

Project Report

on

FINOPS –“Automated Intelligent Document Processing system for Financial operation(Amount Payable)”

By

YOURAJ RANA

Under the Guidance of

Prof. ANIL PHALKE

Master of Computer Application



**P .E. Society's
Modern Institute of Business Studies (Autonomous)
Nigdi, Pune**

2025-2026

1st, Floor, Shop No D-9, Ganeesham Commercial,
Pimple Saudagar, Survey No 21/8 To 21/24 Main BRT Road,
Pimpri Chinchwad, Pune - 411027
Maharashtra, India

+91 860003589 • Email: info@konpo.com • www.konpo.com

KONPO Pvt. Ltd.

TO WHOM IT MAY CONCERN:

MR. YOURAJ RANA

Intern ID: CTI/A3/C1226

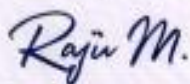
This is to certify that **MR. YOURAJ RANA** (Intern ID: CTI/A3/C1226) successfully completed his internship at **KONPO Pvt. Ltd.** as a Python Developer Intern from **December 20, 2025 to April 7, 2026.**

Mr. Rana was actively involved in Python-based development tasks, contributing to various aspects of application development, and problem-solving. He gained valuable hands-on experience and demonstrated a strong understanding of programming concepts and software development practices.

During his internship, he demonstrated excellent problem-solving skills, strong analytical thinking, and great efficiency in handling assigned tasks, along with a high level of professionalism in all his responsibilities.

We believe that **Mr. Youraj Rana** has the skills and determination to excel in any future role he undertakes, and we extend our best wishes for his continued success.

Sincerely,



Rajiv M.
Chief Technology Officer
KONPO Pvt. Ltd.





Progressive Education Society's
Modern Institute of Business Studies (Autonomous)
Nigdi, Pune-411044

Certificate

This is to certify that the **Online Job Training (OJT-2441)** entitled ***FINOPS “Automated Intelligent Document Processing system for Financial operation(Amount Payable)”*** is submitted by **YOURAJ RANA** a bonafide student of this institute, studying for a Master of Computer Application, has successfully carried out MCA semester-IV. This project report is submitted in partial fulfilment of the **M.C.A. Semester IV Academic Year 2025-26** curriculum as per the Institute norms.

Dr. Maithili Arjunwadkar
Director
PES MIBS

Project Guide

External Examiner

DECLARATION

I, the undersigned, solemnly declare that the report of the project work **FINOPS** is based on my own work carried out during the course of my study under the supervision of **Prof. ANIL PHALKE**. I assert that the statements made and conclusions drawn are an outcome of the project work.

I further declare that to the best of my knowledge and belief, the project report does not contain any part of any work which has been submitted for the award of any other degree/diploma/certificate in this Institute or University or any other University.

YOURAJ RANA

Date:

Place:

ACKNOWLEDGEMENT

It is my proud privilege to express my profound gratitude to the entire management of Progressive Education Society's Modern Institute of Business Studies (Autonomous) and Faculty members of the institute for providing me with the opportunity to avail the excellent facilities and infrastructure of the institute. The knowledge and values inculcated have proved to be of immense help to my career.

I am grateful to **Prof. Dr. Maithili Arjunwadkar (Director, MIBS)** for their astute guidance, constant encouragement and sincere support for this project work.

I express my sincere gratitude to **Prof. Anil Phalke** for their inspiration, constructive suggestion, mastermind analysis and affectionate guidance in my work, without which this project work completion would have been impossible for me. Sincere thanks to all my seniors and colleagues at the company for their support and assistance throughout the project.

I would like to thank all the faculty for providing me with an opportunity to pursue my industrial training, as it is an important part of the MCA course.

INDEX

Chapter	Content	Page Number
Chapter 1	Introduction	8-19
1.1	Company Profile/Institute Profile/Client Profile	8
1.2	Abstract	9
1.3	Existing System and Need for System	10-11
1.4	Scope of System	12-13
1.5	Operating Environment-Hardware and Software	14-15
1.6	Brief Description of Technology Used Operating Systems used (Windows or Unix) Databases	16-19
Chapter 2	Proposed System	20-29
2.1	Study of Similar Systems	22-24
2.2	Feasibility Study	24-25
2.3	Objectives of Proposed System	26-27
2.4	Users of System	28-29
Chapter 3	Analysis and Design	30-48
3.1	System Requirements (Functional and Non-Functional requirements)	30-34
3.2	Entity Relationship Diagram (ERD)(if applicable)	35
3.3	Use Case Diagrams	36
3.4	Class Diagram	37
3.5	Activity Diagram	38

3.6	Table Structure	39-42
3.7	Data Dictionary	43-44
3.8	Sample Input and Output Screens	45-48
Chapter 4	Coding	49-64
4.1	Algorithm	49-54
4.2	Code Snippets	55-64
Chapter 5	Testing	65-67
5.1	Test Strategy	65
5.2	Unit Test Plan	66
5.3	Acceptance Test Plan	66
5.4	TestCase/Test Script	66-67
Chapter 6	Limitations of the Proposed System	65-69
Chapter 7	Proposed Enhancement	70-71
Chapter 8	Conclusion	72-73
Chapter 9	Bibliography	74-75
	Annexure -Progress Sheet	76
	User Manual	77-86

Chapter 1: Introduction

1.1 Company Profile

Konpo Pvt. Ltd. is a privately held Information Technology company based in Pune, Maharashtra, India. The organization specializes in delivering end-to-end software solutions, including full-stack web application development, enterprise software systems, and cloud-based platforms. The company serves multiple domains such as e-commerce, education technology, healthcare, and enterprise resource planning.

Konpo Pvt. Ltd. follows modern software development practices with a strong emphasis on Agile methodology. This approach allows the company to deliver scalable, efficient, and high-quality solutions within defined timelines. The team consists of experienced software engineers, UI/UX designers, and project managers who collaborate to build robust and user-friendly systems.

The organization primarily uses modern technology stacks such as MERN (MongoDB, Express.js, React.js, Node.js), along with Python-based frameworks for data-driven applications. The company also integrates Artificial Intelligence (AI) and cloud technologies into its solutions to meet the evolving needs of businesses.

As part of its academic collaboration, Konpo Pvt. Ltd. provides On-the-Job Training (OJT) programs for students. These programs aim to bridge the gap between theoretical knowledge and real-world application. During this training period, students are exposed to live projects, industry practices, and problem-solving approaches.

The current project titled “FINOPS – Automated Intelligent Document Processing System for Financial Operations (Accounts Payable)” was developed during the OJT under the guidance of technical mentors. The project focuses on automating invoice processing using AI and modern software tools.

1.2 Abstract

In modern organizations, the Accounts Payable process plays a critical role in managing financial transactions. However, most companies still rely on traditional methods involving manual data entry, verification, and communication. These processes are time-consuming, error-prone, and inefficient, leading to financial losses and operational delays.

The FINOPS system is designed to automate the entire invoice processing workflow using Artificial Intelligence and automation techniques. The system accepts invoices in the form of PDFs or images and extracts relevant information such as vendor details, amounts, and item lists using the Google Gemini Vision API.

To prevent duplicate payments, the system uses MD5 cryptographic hashing to generate a unique fingerprint for each uploaded file. This ensures that no invoice is processed more than once. Additionally, the system performs automated validation by verifying mathematical accuracy and checking for policy violations.

In case of discrepancies, an automated email system generates and sends dispute messages to vendors, eliminating manual communication. The system also implements Role-Based Access Control (RBAC) to ensure secure and controlled access for different users.

Overall, the FINOPS system transforms a manual, error-prone process into a fast, secure, and intelligent workflow, improving efficiency and accuracy in financial operations.

1.3 Existing System and Need for System

Prior to the development of the FINOPS system, no unified automated platform existed within the organization to manage the end-to-end Accounts Payable process. Invoice handling was carried out using a fragmented combination of manual data entry, email-based communication, spreadsheet management, and basic accounting tools. This approach lacked integration, intelligence, and automation, resulting in inefficiencies across the financial workflow. The existing system suffered from several critical deficiencies, which are described below:

- **Absence of centralized invoice processing system:** There was no single integrated platform to handle invoice processing. Data was scattered across Excel sheets, emails, and local storage systems. This lack of centralization made it difficult to track invoices, maintain consistency, and retrieve information efficiently when required.
- **Manual data extraction and entry:** Operators were required to manually read invoice documents and enter details such as vendor name, invoice number, and financial amounts into the system. This repetitive process consumed significant time and increased dependency on human effort, making the system inefficient.
- **High probability of human errors:** Since the entire process depended on manual input, there was a high risk of errors such as incorrect data entry, missing fields, or calculation mistakes. Even minor inaccuracies could lead to serious financial discrepancies and affect the reliability of records.
- **No duplicate invoice detection mechanism:** The system lacked the ability to detect duplicate invoices. If the same invoice was submitted more than once, there was no automated way to identify it, which could result in duplicate payments and financial loss.

- Manual verification process (Maker-Checker approach): After data entry, invoices were verified manually by a manager. This involved checking calculations and validating information against the original document. While this added a layer of control, it significantly increased processing time and reduced efficiency.
- Inefficient dispute resolution process: In case of errors or discrepancies, communication with vendors was handled manually through emails. Drafting messages, sending them, and waiting for responses made the process slow and delayed invoice processing.
- Lack of real-time analytics and reporting: The system did not provide real-time insights into financial data. Reports were generated manually, often at the end of a period, which delayed decision-making and reduced visibility into ongoing financial operations.
- No automation in validation and auditing: There was no automated mechanism to verify whether invoice calculations were correct or whether items complied with company policies. All validation was done manually, increasing workload and reducing accuracy.
- Security and access control limitations: The system did not implement role-based access control. There was no proper restriction on user access, and no audit logs were maintained to track user activities, which posed security risks.
- Poor scalability of the system: As the volume of invoices increased, the manual system became difficult to manage. It was not capable of handling large-scale operations efficiently, leading to delays and operational inefficiencies.

The need for a fully automated, intelligent, and centralized financial processing system was therefore clearly established. The proposed FINOPS system addresses these shortcomings by introducing AI-based data extraction, duplicate detection using cryptographic hashing, automated validation, role-based access control, and real-time analytics. This transformation enables a faster, more accurate, and secure Accounts Payable workflow.

1.4 Scope of System

The scope of the FINOPS system defines the functional boundaries and capabilities of the proposed solution. It outlines what the system is designed to achieve and clarifies the features included within the project. The system focuses on automating the Accounts Payable workflow by integrating Artificial Intelligence, data validation mechanisms, and secure user management. The key functionalities within the scope of the system are described below:

- **Intelligent document processing:** The system is capable of accepting invoices in various formats such as PDF files and images. It uses advanced AI-based techniques to extract structured information from unstructured documents, including vendor details, invoice numbers, and financial amounts. This eliminates the need for manual data entry.
- **Duplicate invoice detection:** To prevent duplicate processing, the system generates a unique cryptographic hash (MD5) for every uploaded document. This ensures that the same invoice cannot be processed more than once, thereby reducing the risk of duplicate payments.
- **Automated validation and auditing:** The system automatically verifies the accuracy of invoice data by checking whether the subtotal and tax values match the total amount. It also scans item details to identify any violations of company policies, ensuring compliance without manual intervention.
- **Automated dispute resolution:** In cases where discrepancies are detected, the system generates and sends automated emails to vendors explaining the issue. This reduces manual communication effort and speeds up the resolution process.
- **Role-based access control (RBAC):** The system supports multiple user roles, primarily Operators and Admins. Operators are responsible for uploading and processing invoices, while Admins have access to analytics and audit logs. This ensures secure and controlled access to system functionalities.

- **Real-time analytics and reporting:** The system provides a dashboard for administrators to view financial data in real time. It allows monitoring of total expenses, flagged invoices, and processing status, enabling better decision-making.
- **Centralized data management:** All invoice data, user information, and audit logs are stored in a structured database. This centralization improves data consistency, simplifies retrieval, and ensures better organization of financial records.
- **Data export functionality:** The system allows administrators to export processed data into formats such as Excel or CSV. This facilitates external reporting, auditing, and integration with other financial systems.
- **Secure audit logging:** Every action performed within the system, such as login, invoice upload, and email communication, is recorded in an audit log. This ensures transparency and accountability.
- **User-friendly interface:** The system provides a simple and intuitive interface that allows users to perform operations such as uploading invoices and viewing results without requiring extensive technical knowledge.

The system, however, operates within certain defined boundaries. It does not include direct bank transaction processing, offline functionality, or integration with external ERP systems in its current version. These features are considered as potential future enhancements.

The scope of the FINOPS system focuses on automating and optimizing the invoice processing workflow while ensuring accuracy, security, and efficiency. It provides a comprehensive solution within defined boundaries and lays the foundation for further expansion and integration.

1.5 Operating Environment - Hardware and Software

The operating environment of the FINOPS system defines the hardware and software requirements necessary for the proper functioning of the application. Since the system is designed as a web-based solution integrated with cloud-based AI services, it does not require high-end infrastructure but does depend on a stable computing and network environment. The operating environment is categorized into hardware and software requirements as described below:

1.5.1 Hardware Requirements

- **Processor:** The system requires a minimum of an Intel Core i3 or equivalent processor for basic operations. For better performance and faster data processing, especially when handling multiple invoices or running analytics, an Intel Core i5 or higher is recommended. The processor is responsible for executing application logic and managing data operations efficiently.
- **Random Access Memory (RAM):** A minimum of 4 GB RAM is required for smooth execution of the application. However, 8 GB RAM is recommended to ensure better performance while handling multiple tasks such as data processing, running the web interface, and interacting with the database simultaneously.
- **Storage:** The system requires at least 20 GB of free storage space. This storage is used for installing the operating system, application files, database storage, and temporary storage of uploaded invoice documents.
- **Network Connectivity:** A stable and active internet connection is essential for the system. Since the application relies on cloud-based services such as AI APIs and email servers, uninterrupted network connectivity is required to ensure proper functioning of features like data extraction and automated email communication.

- **Client Devices:** The system can be accessed through any standard computing device such as a desktop, laptop, or tablet. The application is browser-based, so no specialized hardware is required on the client side.

1.5.2 Software Requirements

- **Operating System:** The system is platform-independent and can run on various operating systems such as Windows 10/11 or Linux distributions like Ubuntu. This flexibility allows the application to be deployed in different environments without compatibility issues.
- **Programming Language:** The core logic of the system is implemented using Python. Python is chosen due to its simplicity, flexibility, and strong support for data processing, AI integration, and backend development.
- **Web Framework:** The system uses Streamlit as the web framework to build the user interface. Streamlit enables rapid development of interactive web applications without requiring extensive frontend coding.
- **Database Management System:** MySQL is used as the relational database management system to store structured data such as user details, invoice records, and audit logs. It ensures data integrity, consistency, and efficient data retrieval.
- **Web Browser:** The application can be accessed through any modern web browser such as Google Chrome, Microsoft Edge, or Mozilla Firefox. A browser is required for users to interact with the system interface.
- **External Services and APIs:** The system integrates with external services such as Google Gemini API for AI-based data extraction and SMTP servers for sending automated emails. These services enhance the functionality of the system and enable intelligent automation.).

1.6 Brief Description of Technology Used

To develop the FINOPS system, a modern, lightweight, and efficient technology stack was selected. The chosen technologies ensure smooth integration between different modules such as invoice processing, data extraction, validation, and reporting. Each component of the technology stack plays a specific role in achieving automation, accuracy, and scalability in the system. The technologies used are described below:

• Python (Core Engine):

Python serves as the backbone of the entire system and is responsible for implementing the core business logic. It is widely used due to its simplicity, readability, and extensive support for libraries. In the FINOPS system, Python handles critical operations such as file processing, hash generation, API communication, validation logic, and database interaction.

One of the major advantages of Python is its rich ecosystem of libraries, which simplifies complex tasks such as data manipulation and automation. Additionally, Python provides seamless integration with external APIs, making it ideal for implementing AI-based features like invoice data extraction. Its flexibility and modular structure allow easy maintenance and future enhancements of the system.

• Streamlit (Frontend / Web Framework):

Streamlit is used as the frontend framework to build the user interface of the application. Unlike traditional web development, which requires separate technologies such as HTML, CSS, and JavaScript, Streamlit allows the entire application to be developed using Python.

It provides built-in components such as file upload widgets, buttons, dashboards, and data display elements, which are essential for the FINOPS system. The Operator can easily upload invoices, view extracted data, and interact with the system through a clean and simple interface. Similarly, the Admin can access dashboards and reports without complexity.

Streamlit enables rapid development of web applications and ensures that the system remains lightweight and easy to use, making it suitable for both technical and non-technical users.

- **Google Gemini Vision API (AI-Based Data Extraction):**

The Google Gemini Vision API acts as the intelligent component of the system and is responsible for extracting structured data from unstructured invoice documents. Unlike traditional OCR systems, which only convert images into text, Gemini is a multimodal generative AI that understands the context and layout of documents.

This capability allows the system to accurately identify important fields such as vendor name, invoice date, subtotal, tax, and total amount, regardless of how the invoice is formatted. Even in cases where invoices have complex layouts or tables, the API can extract meaningful information with high accuracy.

By using this AI-based approach, the system eliminates the need for manual data entry and significantly improves efficiency and accuracy in invoice processing.

- **MySQL (Database Management System):**

MySQL is used as the relational database for storing all structured data in the system. It manages key tables such as users, invoices, and audit_logs, ensuring that all information is stored in an organized and consistent manner.

The database follows normalization principles (up to 3NF), which reduces redundancy and improves data integrity. MySQL also supports ACID properties (Atomicity, Consistency, Isolation, Durability), ensuring reliable transaction handling.

Through structured queries, the system can efficiently store, retrieve, and update data. This makes MySQL a suitable choice for managing financial records and maintaining long-term data consistency.

- **Cryptography (hashlib for MD5 Hashing):**

The system uses Python's built-in hashlib library to generate MD5 hash values for uploaded invoice files. Each file is converted into a unique 32-character hash string, which acts as its digital fingerprint.

This hash is stored in the database and used to detect duplicate invoices. If a file with the same hash is uploaded again, the system identifies it as a duplicate and prevents further processing. This mechanism ensures data integrity and helps in avoiding duplicate payments.

Although MD5 is not used for high-level security, it is efficient and sufficient for file identification and duplicate detection in this context.

- **SMTP (smtplib for Email Communication):**

The Simple Mail Transfer Protocol (SMTP) is used to enable automated email communication within the system. Using Python's smtplib library, the system connects to email servers such as Gmail and sends emails programmatically.

This feature is used in the dispute resolution process, where the system automatically generates and sends emails to vendors when discrepancies are detected in invoices. The use of SMTP reduces manual effort and ensures faster communication.

Secure transmission is maintained using protocols such as TLS, ensuring that email communication is safe and reliable.

- **Pandas & ExcelWriter (Data Processing and Reporting):**

Pandas is a powerful Python library used for data manipulation and analysis. In the FINOPS system, it is used to process invoice data stored in the database and convert it into structured formats.

The ExcelWriter module is used in combination with Pandas to generate downloadable Excel reports. These reports are well-formatted and can be used by administrators for analysis, auditing, and record-keeping.

This functionality enhances the reporting capabilities of the system and provides users with an easy way to export and analyze financial data.

The FINOPS system is built using a well-integrated and efficient technology stack that combines Python-based development, AI-powered data extraction, and database management. Each technology plays a specific role in ensuring automation, accuracy, and scalability. The use of modern tools such as Streamlit and Gemini API enables rapid development and intelligent processing, making the system reliable and suitable for real-world financial operations.

Chapter 2: Proposed System

2.1 Study of Similar Systems

Before designing and developing the proposed FINOPS system, a detailed study of existing systems and tools used in financial operations and invoice processing was conducted. This study helps in understanding current industry practices, identifying limitations, and determining how the proposed system can provide improved functionality. The analysis includes manual systems, traditional OCR tools, and enterprise financial management solutions.

• Manual Accounting Systems (e.g., Excel, Tally):

In many small and medium-sized organizations, invoice processing is handled using basic tools such as Microsoft Excel or accounting software like Tally. In these systems, operators manually read invoices and enter relevant details into spreadsheets or accounting modules. While these tools are easy to use and cost-effective, they lack automation and intelligence. They do not provide features such as automatic data extraction, validation, or duplicate detection. As a result, they are highly dependent on human effort and are prone to errors, making them inefficient for large-scale operations.

• Traditional OCR-Based Systems:

Optical Character Recognition (OCR) tools are used to convert scanned documents and images into machine-readable text. Tools such as ABBYY FineReader and Tesseract OCR are commonly used for this purpose. While OCR can extract text from invoices, it operates on a basic pattern recognition approach and lacks contextual understanding. If the layout of the invoice changes or the formatting is inconsistent, OCR systems may extract incorrect or incomplete data. Additionally, these systems do not perform validation or business logic checks, limiting their effectiveness in financial workflows.

- **Enterprise Resource Planning (ERP) Systems (e.g., SAP, Oracle Financials):**

Large organizations use ERP systems such as SAP and Oracle Financials to manage financial operations. These systems provide integrated modules for accounting, procurement, and invoice management. While they offer advanced features and high scalability, they are expensive to implement and require specialized training. Customizing these systems to include specific business rules or automation features can be complex and costly. Moreover, smaller organizations may find these solutions difficult to adopt due to high infrastructure and licensing costs.

- **Expense Management Systems (e.g., SAP Concur, Zoho Expense):**

Expense management platforms are designed to handle employee expenses and reimbursements. Tools like SAP Concur and Zoho Expense offer features such as receipt scanning, expense tracking, and reporting. Although they include some level of automation, their primary focus is on employee expense management rather than invoice processing. They may not provide comprehensive validation, duplicate detection, or policy enforcement required for Accounts Payable operations.

- **Limitations of Existing Systems:**

The study of existing systems reveals several common limitations. Most systems either rely heavily on manual processes or provide limited automation without intelligence. Traditional tools lack the ability to understand document context, perform automatic validation, or prevent duplicate processing. Enterprise systems, while powerful, are costly and complex to implement. As a result, there is a gap between affordability, automation, and intelligence in current solutions.

The analysis of similar systems clearly indicates the need for a solution that combines automation, intelligence, and cost-effectiveness. The proposed FINOPS system addresses these gaps by integrating AI-based document processing, automated validation, duplicate detection, and real-time analytics into a single unified platform. This approach ensures improved efficiency, accuracy, and scalability compared to existing systems.

2.2 Feasibility Study

A feasibility study is an essential step in system development that evaluates whether the proposed system is practical and achievable within given constraints. It helps in determining whether the system can be successfully implemented from technical, economic, and operational perspectives. The feasibility analysis of the FINOPS system is described below:

2.2.1 Technical Feasibility

Technical feasibility examines whether the required technology, tools, and infrastructure are available to develop and implement the system successfully.

- **Availability of Technology**

The FINOPS system is built using widely available and well-supported technologies such as Python, Streamlit, MySQL, and cloud-based AI services. These technologies are open-source or easily accessible, making them suitable for development without any major technical barriers.

- **Integration with AI Services:**

The system integrates with the Google Gemini Vision API for intelligent document processing. This API provides advanced capabilities for extracting structured data from invoices, eliminating the need to build complex machine learning models from scratch. This makes the system technically feasible and efficient.

- **Development Tools and Libraries**

Python provides a rich ecosystem of libraries such as Pandas for data processing and smtplib for email communication. These tools simplify development and enable implementation of complex functionalities with minimal effort.

- **Hardware and Infrastructure Requirements**

The system does not require high-end hardware since most processing is handled through cloud services. A standard computer with internet connectivity is sufficient to run the application, making deployment easy and cost-effective.

- **Security Implementation:**

The system incorporates standard security practices such as password hashing, role-based access control, and secure API communication. These mechanisms can be implemented using existing tools and libraries, ensuring a secure system environment.

2.2.2 Economic Feasibility

Economic feasibility evaluates whether the benefits of the system justify its development and operational costs.

- **Low Development Cost**

The FINOPS system is developed using open-source technologies such as Python, Streamlit, and MySQL. This eliminates licensing costs and reduces the overall development expense.

- **Minimal Infrastructure Cost**

Since the system relies on cloud-based services and does not require dedicated servers, the infrastructure cost is minimal. Most services operate on a pay-as-you-use model, making them affordable for small and medium-sized organizations.

- **Reduction in Operational Cost**

Automation reduces the need for manual labor in invoice processing. This decreases the workload on employees and reduces operational costs associated with human effort and error correction.

- **Prevention of Financial Loss**

By detecting duplicate invoices and minimizing errors, the system helps prevent financial losses. This contributes to a higher return on investment over time.

- **Scalability Without High Cost**

The system can handle increasing volumes of invoices without significant additional cost, making it economically sustainable for growing organizations.

2.2.3 Operational Feasibility

Operational feasibility determines whether the system will be accepted and effectively used by its intended users.

- **Ease of Use**

The system is designed with a simple and intuitive user interface, allowing users to perform tasks such as uploading invoices and viewing results without requiring extensive training.

- **Compatibility with Existing Workflow**

The FINOPS system follows the existing “Maker-Checker” workflow by assigning roles such as Operator and Admin. This ensures that users can easily adapt to the new system without major changes in their working style.

- **Reduced Training Requirement**

Since the interface is user-friendly and similar to common web applications, users can quickly learn to operate the system. This reduces the need for extensive training programs.

- **Improved Efficiency and Productivity**

Automation of repetitive tasks reduces workload and increases productivity. Users can focus on higher-level decision-making rather than manual data entry and verification.

- **User Acceptance**

The system addresses major pain points such as time consumption, errors, and lack of automation. This increases the likelihood of acceptance among users and ensures smooth implementation.

2.3 Objectives of Proposed System

The primary objective of the FINOPS system is to develop an intelligent, automated, and efficient solution for managing the Accounts Payable process. The system aims to overcome the limitations of traditional manual methods by integrating modern technologies such as Artificial Intelligence, data validation mechanisms, and secure user management. The key objectives of the proposed system are described below:

- **To automate invoice data extraction**

The system aims to eliminate manual data entry by using AI-based document processing. It automatically extracts important information such as vendor name, invoice date, subtotal, tax, and total amount from invoices in PDF or image format. This reduces human effort and increases processing speed.

- **To prevent duplicate invoice processing**

One of the major objectives is to ensure that the same invoice is not processed more than once. The system uses cryptographic hashing techniques (MD5) to generate a unique identifier for each uploaded file, thereby preventing duplicate entries and avoiding financial losses.

- **To perform automated validation and auditing**

The system is designed to automatically verify the accuracy of invoice data by checking mathematical calculations and ensuring compliance with company policies. This reduces the need for manual verification and improves data accuracy.

- **To reduce processing time and improve efficiency**

By automating repetitive tasks such as data extraction, validation, and communication, the system significantly reduces the time required to process each invoice. This improves overall efficiency and productivity.

- **To enable automated dispute resolution**

The system generates and sends automated emails to vendors in case of discrepancies. This reduces manual communication effort and ensures faster resolution of issues.

- **To provide real-time analytics and reporting**

The system offers a dashboard for administrators to monitor financial data, including total expenses, flagged invoices, and processing status. This helps in better decision-making and financial planning.

- **To implement secure role-based access control**

The system ensures that only authorized users can access specific functionalities. Operators and Admins have different access levels, which enhances data security and prevents unauthorized actions.

- **To maintain audit logs for transparency**

Every action performed within the system is recorded in audit logs. This ensures accountability and provides a clear record of all activities for future reference and auditing purposes.

- **To ensure centralized data management**

The system stores all invoice-related data in a structured database, ensuring consistency, easy retrieval, and better organization of records.

- **To develop a scalable and flexible system**

The system is designed to handle increasing volumes of invoices and support future enhancements. It provides a flexible architecture that can be extended with additional features as required.

The objectives of the proposed FINOPS system focus on improving efficiency, accuracy, security, and scalability in financial operations. By achieving these objectives, the system provides a reliable and modern solution for automating the Accounts Payable process.

2.4 Users of System

The FINOPS system is designed to operate using a Role-Based Access Control (RBAC) mechanism, where different users are assigned specific roles and permissions. This ensures that each user can access only the functionalities relevant to their responsibilities. The system defines two primary types of users: Operator and Administrator. These roles are clearly separated to maintain security, efficiency, and proper workflow management within the Accounts Payable process.

• Operator (Maker / Data Processing User):

The Operator is responsible for handling the day-to-day operations of the system. This role typically includes data entry staff or junior accountants who are involved in invoice processing. The primary responsibility of the Operator is to upload invoice documents into the system in the form of PDFs or images. Once the invoice is uploaded, the system automatically extracts relevant data using AI, and the Operator can review the extracted information.

The Operator is also responsible for verifying the system output at a basic level, such as checking extracted values and identifying flagged errors. In cases where discrepancies are detected, such as mathematical mismatches or policy violations, the Operator can initiate the automated dispute process. This involves triggering the system to send an email to the vendor for correction.

However, the Operator has limited access within the system. They are not allowed to view overall financial analytics, system logs, or sensitive data. This restriction ensures that the Operator focuses only on operational tasks and does not interfere with management-level functionalities.

- **Administrator (Admin / Management User)**

The Administrator is responsible for monitoring and controlling the overall functioning of the system. This role is typically assigned to finance managers or senior-level personnel. The Admin has access to high-level features such as analytics dashboards, audit logs, and financial summaries.

The Administrator can view processed invoices, monitor system performance, and analyze financial data through visual reports. They are also responsible for ensuring that all operations are carried out correctly and that no unauthorized activities take place within the system. The audit logs available to the Admin provide a complete record of user actions, which helps in maintaining transparency and accountability.

Unlike Operators, Administrators do not perform routine tasks such as uploading invoices. Their role is focused on supervision, decision-making, and maintaining system integrity.

The clear separation of roles between Operator and Administrator ensures that the FINOPS system operates efficiently and securely. By restricting access based on user roles, the system prevents unauthorized actions and maintains proper control over financial operations.

Chapter 3: Analysis and Design

3.1 System Requirements (Functional and Non-Functional Requirements)

A. Functional Requirements (FRs)

Functional requirements define the specific operations and services that the FINOPS system must perform. These requirements describe how the system behaves and what functionalities it provides to users. The following functional requirements are identified based on the system design and scope:

- **FR-01: User Authentication and Role Management**

The system shall provide a secure login mechanism for users. It shall differentiate between Operator and Administrator roles based on stored user credentials. After successful authentication, the system shall grant access to functionalities according to the assigned role. Unauthorized users shall not be allowed to access restricted features.

- **FR-02: Invoice Upload and Processing**

The system shall allow the Operator to upload invoice documents in the form of PDF files or images. Upon upload, the system shall accept the file and initiate the processing pipeline for further operations such as duplicate detection and data extraction.

- **FR-03: Duplicate Invoice Detection**

The system shall generate a unique MD5 hash value for each uploaded invoice file. It shall compare this hash value with existing records in the database. If a matching hash is found, the system shall identify the file as a duplicate and prevent further processing.

- FR-04: AI-Based Data Extraction

The system shall extract relevant data from uploaded invoices using the Google Gemini Vision API. It shall retrieve key information such as vendor name, vendor email, invoice date, subtotal, tax amount, and total amount. The extracted data shall be displayed to the Operator for review.

- FR-05: Automated Mathematical Validation

The system shall automatically verify the correctness of invoice calculations. It shall compute the expected total by adding subtotal and tax and compare it with the extracted total amount. If a mismatch is found, the system shall flag the invoice as a “Math Error.”

- FR-06: Policy Compliance Check

The system shall scan the extracted item details and compare them against predefined company policies. If any restricted item is detected, the system shall flag the invoice as a “Policy Violation.”

- FR-07: Invoice Status Management

The system shall assign a status to each processed invoice based on validation results. Possible statuses include “Approved,” “Flagged (Math Error),” and “Flagged (Policy Violation).” The status shall be stored in the database and displayed to the user.

- FR-08: Automated Dispute Email Generation

The system shall allow the Operator to generate and send a dispute email for flagged invoices. It shall automatically create an email containing details of the detected error and send it to the vendor using SMTP protocol.

- FR-09: Data Storage and Management

The system shall store all processed invoice data, including file details, extracted information, hash values, and status, in the database. It shall maintain proper relationships between users and invoices.

- FR-10: Audit Logging

The system shall record all critical user actions such as login, invoice upload, and email sending in an audit log. Each log entry shall include details such as user ID, action performed, and timestamp.

- FR-11: Admin Dashboard and Analytics

The system shall provide an Administrator dashboard to view financial data and system statistics. It shall display information such as total invoices processed, number of approved and flagged invoices, and overall financial summaries.

- FR-12: Data Export Functionality

The system shall allow the Administrator to export invoice data into standard formats such as Excel or CSV. This feature shall support external auditing and reporting requirements.

- FR-13: Role-Based Access Restriction

The system shall restrict access to functionalities based on user roles. Operators shall not have access to analytics dashboards or audit logs, while Administrators shall not perform invoice upload operations.

The above functional requirements define the core operations of the FINOPS system. These requirements ensure that the system performs all necessary tasks efficiently while maintaining accuracy, security, and proper workflow management.

B. Non-Functional Requirements (NFRs)

Non-functional requirements define the quality attributes and constraints of the FINOPS system. These requirements do not describe specific functionalities but focus on how the system performs, including aspects such as performance, security, usability, and reliability. The following non-functional requirements are identified for the system:

- NFR-01: Performance Requirement

The system shall process uploaded invoices within a reasonable time. Data extraction using the AI API and validation operations should be completed efficiently so that users do not experience significant delays. The system should handle multiple invoice uploads without noticeable performance degradation.

- NFR-02: Reliability Requirement

The system shall operate consistently without failures during normal usage. It should ensure that all operations such as invoice processing, validation, and data storage are performed accurately. In case of any failure, the system should prevent data loss and maintain data integrity.

- NFR-03: Usability Requirement

The system shall provide a simple and user-friendly interface. Users should be able to upload invoices, view results, and perform actions without requiring extensive technical knowledge. The interface should be intuitive and easy to navigate.

- NFR-04: Security Requirement

The system shall ensure secure access through authentication mechanisms. User credentials must be protected, and role-based access control shall be implemented to restrict access to sensitive functionalities. Data stored in the system should be protected from unauthorized access.

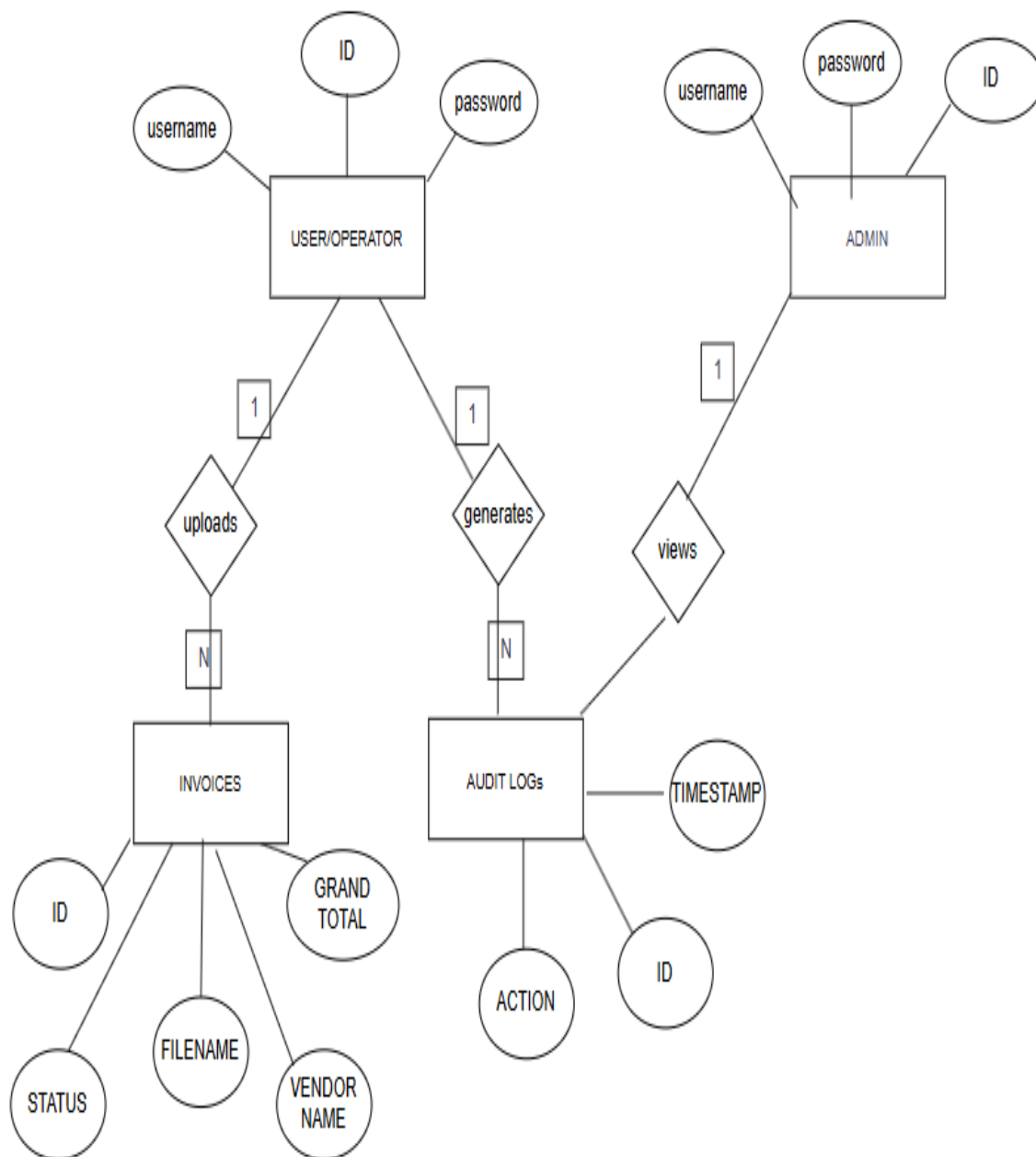
- NFR-05: Scalability Requirement

The system shall be capable of handling an increasing number of invoices as the organization grows. It should support expansion without significant changes to the system architecture or performance issues.

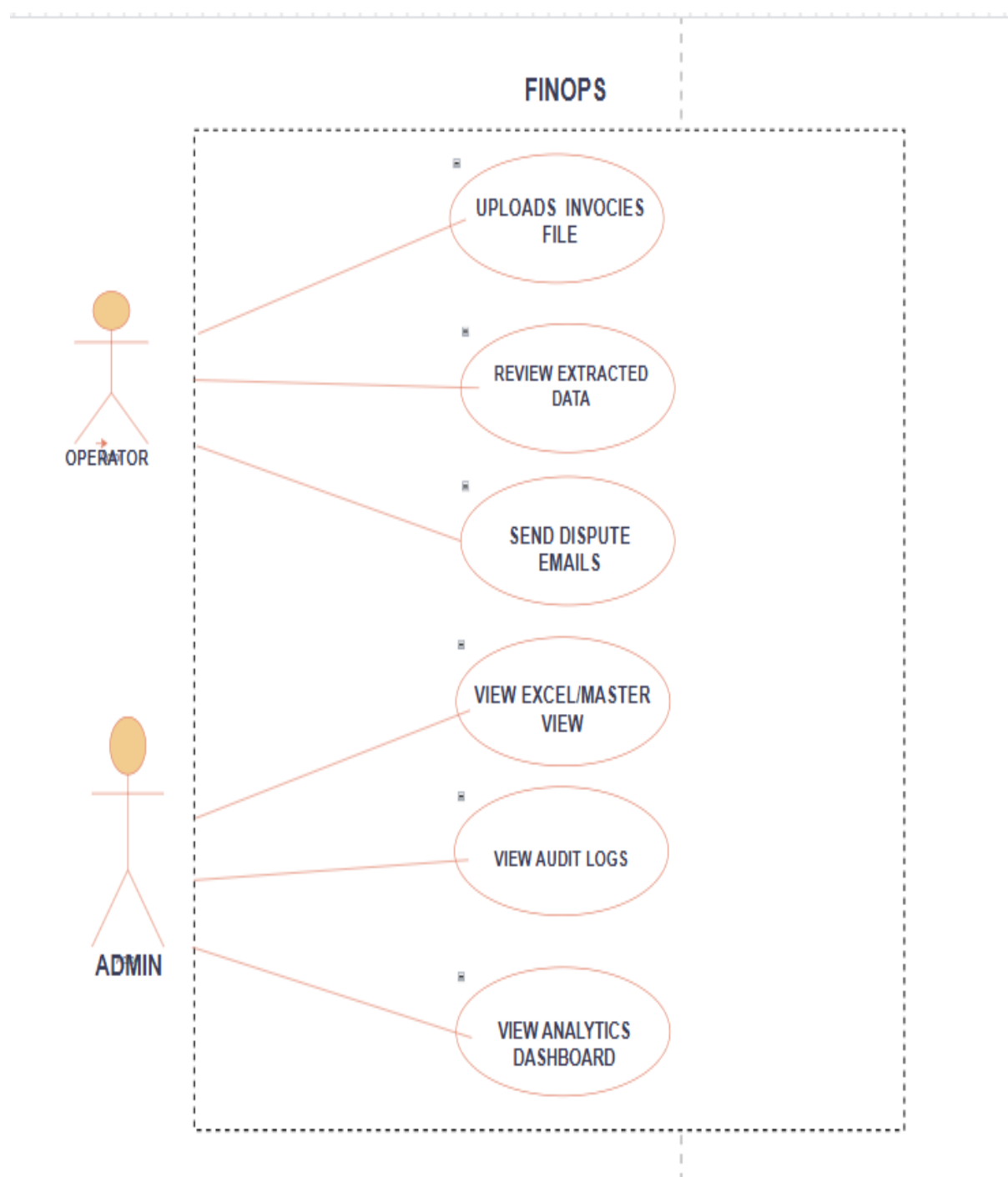
- NFR-06: Availability Requirement

The system shall be available to users whenever required, provided there is internet connectivity. Since it depends on web-based services, the system should ensure maximum uptime and accessibility.

3.2 Entity Relationship Diagram

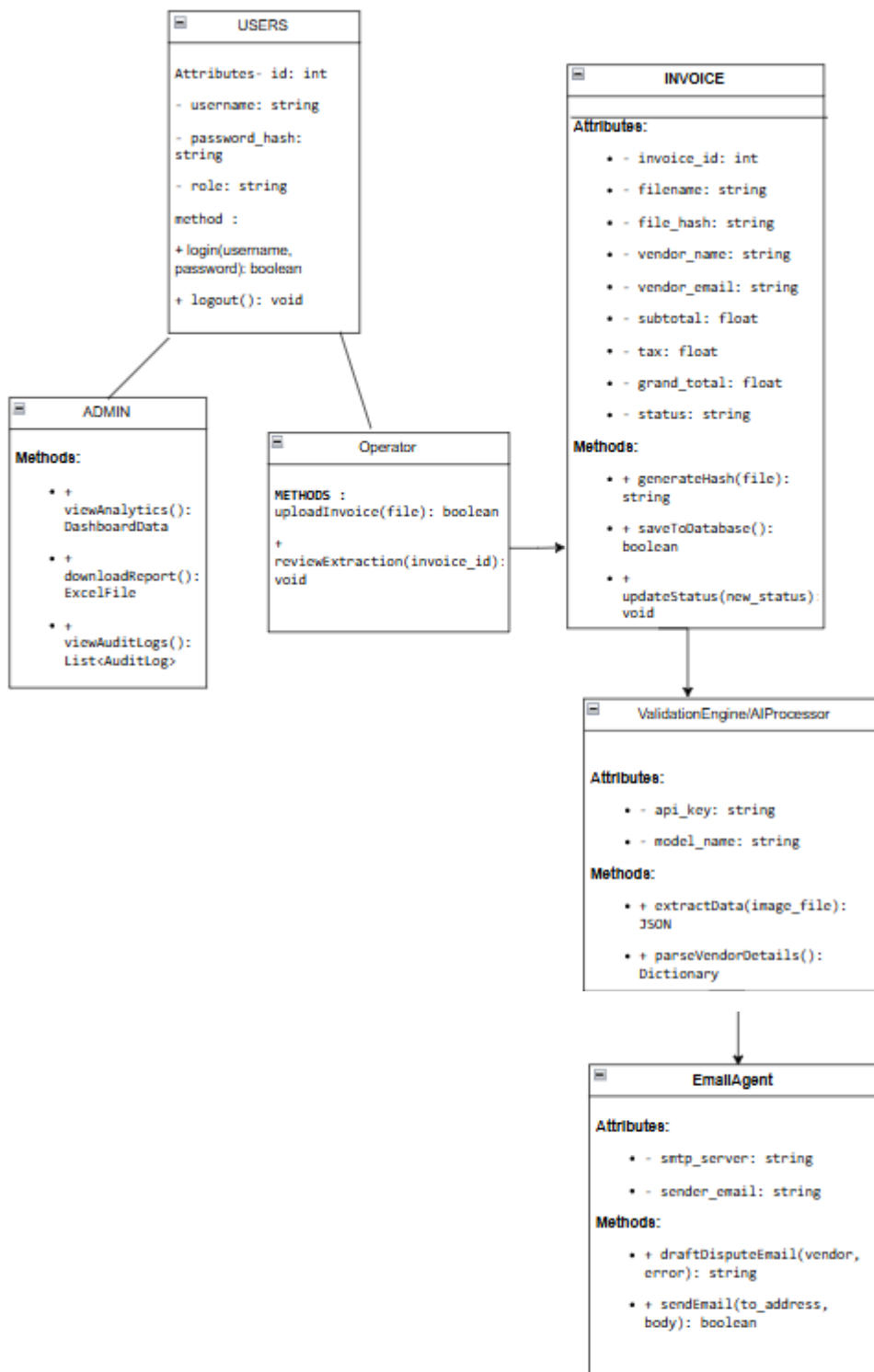


3.3 Use Case Diagrams

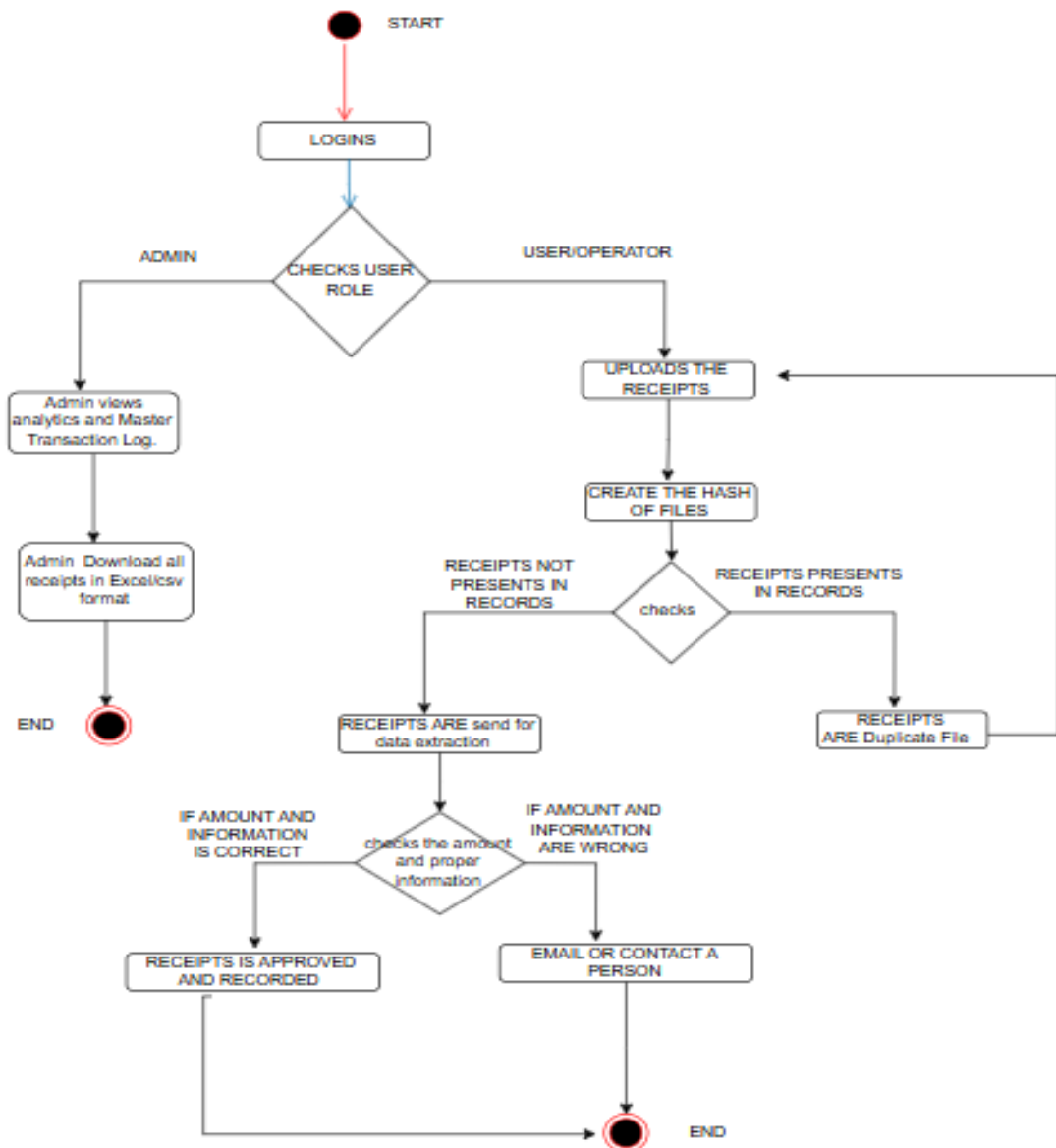


3.4 Class Diagram:

CLASS DIAGRAM



3.5 Activity Diagram



3.6 Table Structure

- The database architecture for the FINOPS Automation System is designed using MySQL to ensure efficient data storage, security, and scalability. To maintain data integrity and eliminate redundancy, the database is normalized up to the Third Normal Form (3NF). This ensures that each piece of data is stored only once and dependencies are properly maintained.
- The system is built upon three core tables: Users, Invoices, and Audit Logs, which are interconnected using foreign key constraints. These tables collectively manage authentication, invoice processing, and system activity tracking.

3.6.1 Users Table

- The Users table acts as the central authentication hub of the system. It stores all user-related information, including login credentials and role details, which are essential for implementing Role-Based Access Control (RBAC).
 - Primary Key: id
 - Relationships:

One user can upload multiple invoices (1:N relationship)

One user can generate multiple audit log entries (1:N relationship)

Fields Description:

Attribute Name	Data Type	Constraints	Description
id	INT	Primary Key, Auto Increment	Unique identifier for each user
username	VARCHAR	NOT NULL, UNIQUE	Stores the login username of the user
password_hash	VARCHAR	NOT NULL	Stores the encrypted password for secure authentication
role	ENUM	NOT NULL	Defines user role (Operator or Admin), controlling system access

Attribute Name	Data Type	Constraints	Description
email	VARCHAR	—	Stores the email address of the user
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Records the date and time of user registration

3.6.2 Invoices Table

- The Invoices table is the main transaction table of the system. It stores both the metadata of uploaded invoice files and the financial data extracted using AI.
 - Primary Key: id
 - Foreign Key: uploaded_by (References users.id)
- Fields Description:

Attribute Name	Data Type	Key/ Constraint	Description
id	INT	Primary Key	Unique identifier for each invoice
uploaded_by	INT	Foreign Key	References the user who uploaded the invoice
file_name	VARCHAR	—	Stores the name of the uploaded file
file_hash	VARCHAR	UNIQUE	Stores the MD5 hash value used for duplicate detection
vendor_name	VARCHAR	—	Stores the vendor name extracted from the invoice
vendor_email	VARCHAR	—	Stores the vendor email address
invoice_date	DATE	—	Represents the date mentioned on the invoice
subtotal	DECIMAL	—	Stores the subtotal amount
tax	DECIMAL	—	Stores the tax amount

Attribute Name	Data Type	Key/ Constraint	Description
total	DECIMAL	—	Stores the final total amount
status	VARCHAR	—	Indicates processing result (Approved, Math Error, Policy Violation)
created_at	TIMESTAMP	—	Records when the invoice was uploaded
currency	VARCHAR(10)	—	Stores the currency of the invoice (e.g., USD, INR)
amount_inr	DECIMAL(15,2)	—	Stores the converted invoice amount in Indian Rupees (INR)

3.6.3 Audit Logs Table

The Audit Logs table serves as the security and monitoring layer of the system. It maintains a record of all critical user actions to ensure transparency, accountability, and traceability.

Primary Key: id

Foreign Key: user_id (References users.id)

Relationships:

Each log entry is associated with one user

A user can generate multiple log entries

❖ Fields Description:

Attribute Name	Data Type	Key /Constraint	Description
id	INT	Primary Key	Unique identifier for each log entry

Attribute Name	Data Type	Key /Constraint	Description
user_id	INT	Foreign Key	References the user who performed the action
action	VARCHAR	—	Describes the activity (e.g., login, invoice upload, email sent)
timestamp	TIMESTAMP	—	Records the exact date and time of the action

3.6.4 Relationships Overview

The relationship between tables ensures proper data organization and integrity:

A User → Invoices relationship is one-to-many (1:N)

A User → Audit Logs relationship is one-to-many (1:N)

Each invoice and audit log is linked to a specific user through foreign keys

- This relational structure ensures that all data is consistent, traceable, and efficiently managed. The database structure of the FINOPS system is designed to be secure, normalized, and efficient. By organizing data into three interconnected tables — Users, Invoices, and Audit Logs — the system ensures proper management of authentication, transaction processing, and activity tracking. The use of foreign keys and normalization techniques enhances data integrity and reduces redundancy.

3.7 Data Dictionary

The Data Dictionary provides a detailed description of all database tables, including their fields, data types, constraints, and purpose. It helps in understanding the structure of the database and ensures clarity in data management.

1. Data Dictionary for USER Table

Attribute Name	Data Type	Size	Constraints	Description
Id	INT	—	Primary Key, Auto Increment	Unique identifier for each user
username	VARCHAR	50	NOT NULL, UNIQUE	Username used for login
password_hash	VARCHAR	255	NOT NULL	Encrypted password for authentication
role	ENUM	—	NOT NULL	Defines user role (Operator/Admin)
email	VARCHAR	100	—	Stores user email address
created_at	TIMESTAMP	—	DEFAULT CURRENT_TIMESTAMP	Stores account creation time

2. Data Dictionary for invoices Table

Attribute Name	Data Type	Size	Constraints	Description
Id	INT	—	Primary Key, Auto Increment	Unique identifier for each invoice
uploaded_by	INT	—	Foreign Key (users.id)	References user who uploaded invoice
file_name	VARCHAR	255	NOT NULL	Name of uploaded file
file_hash	VARCHAR	255	UNIQUE	MD5 hash for duplicate detection
vendor_name	VARCHAR	100	—	Vendor name extracted from invoice
vendor_email	VARCHAR	100	—	Vendor email address
invoice_date	DATE	—	—	Date of invoice
subtotal	DECIMAL	10,2	—	Subtotal amount
Tax	DECIMAL	10,2	—	Tax amount

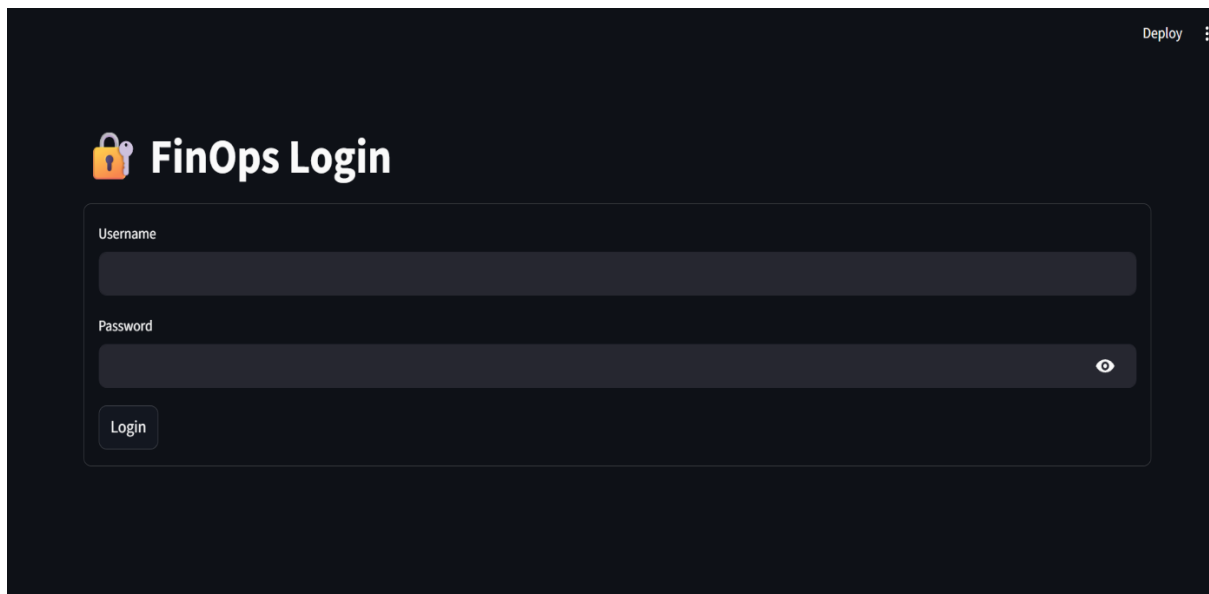
Attribute Name	Data Type	Size	Constraints	Description
total	DECIMAL	10,2	—	Total amount
status	VARCHAR	50	—	Status (Approved / Flagged)
created_at	TIMESTAMP	—	DEFAULT CURRENT_TIMESTAMP	Invoice upload time
currency	VARCHAR	10	—	Stores the currency of the invoice (e.g., USD, INR)
amount_inr	DECIMAL	15,2	—	Stores the converted amount in Indian Rupees (INR)

3 .Data Dictionary for AUDIT_LOGS Table

Attribute Name	Data Type	Size	Constraints	Description
Id	INT	—	Primary Key, Auto Increment	Unique identifier for log
user_id	INT	—	Foreign Key (users.id)	References user performing action
action	VARCHAR	255	NOT NULL	Action performed (login, upload, email)
timestamp	TIMESTAMP	—	DEFAULT CURRENT_TIMESTAMP	Time of action

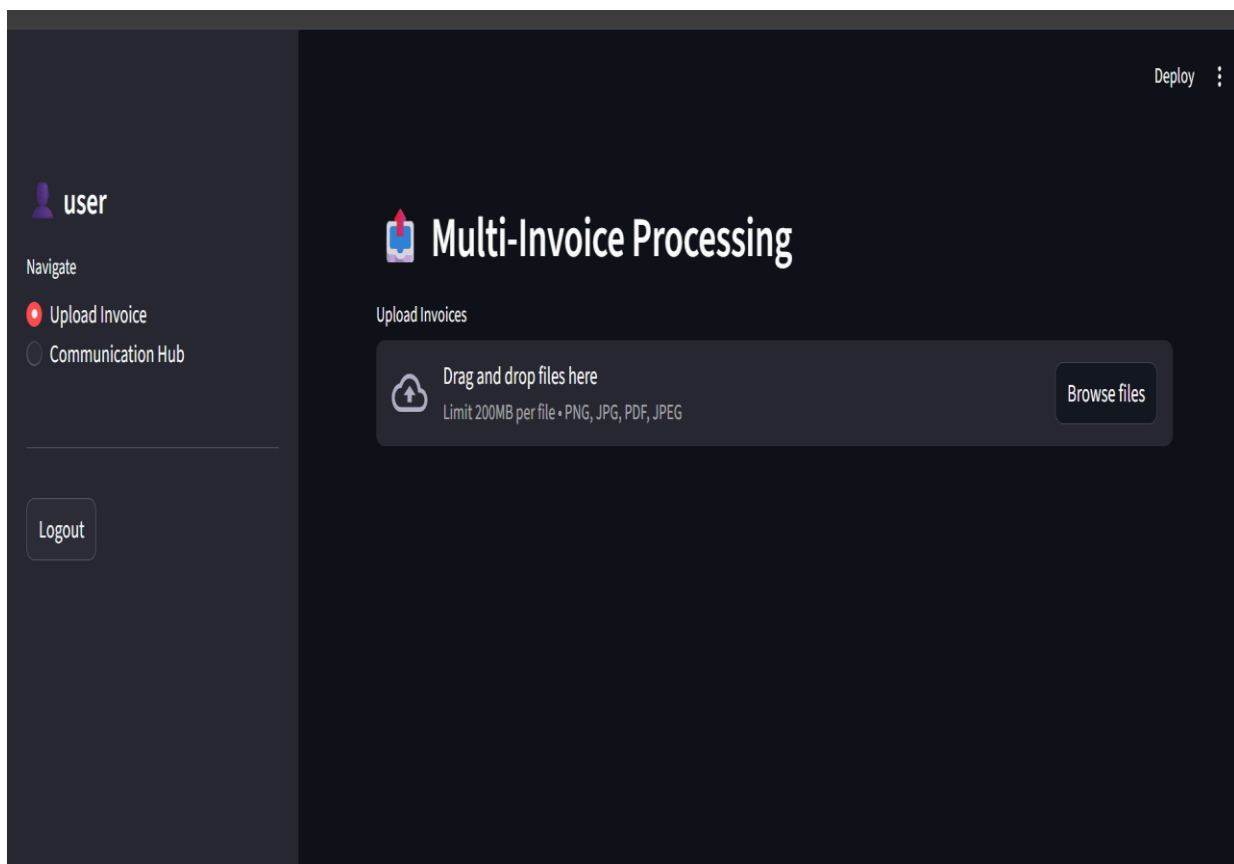
3.8 Sample Input and Output Screens:

3.8.1 login screen:



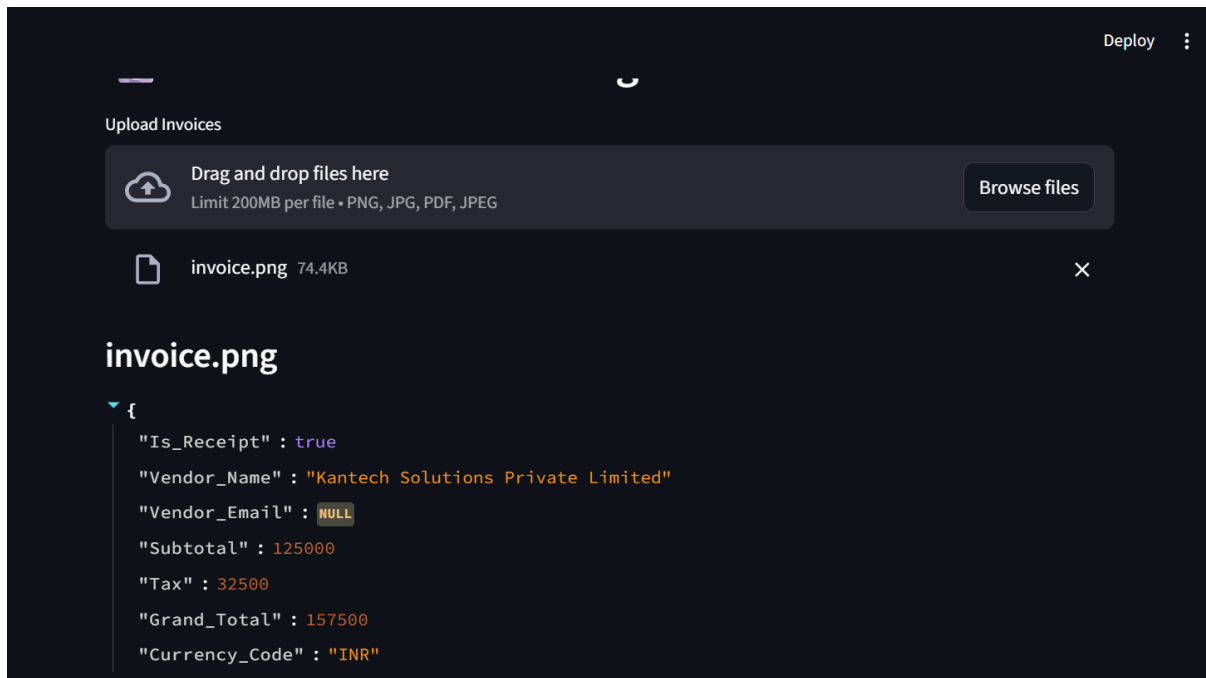
The image shows the 'FinOps Login' screen. It has a dark background. At the top right, there is a 'Deploy' button with a three-dot menu icon. The main heading is 'FinOps Login' with a padlock icon. Below the heading, there are two input fields: 'Username' and 'Password'. The 'Password' field has a toggle icon (an eye) to its right. At the bottom left of the form, there is a 'Login' button.

3.8.2 OPERATOR SCREEN :

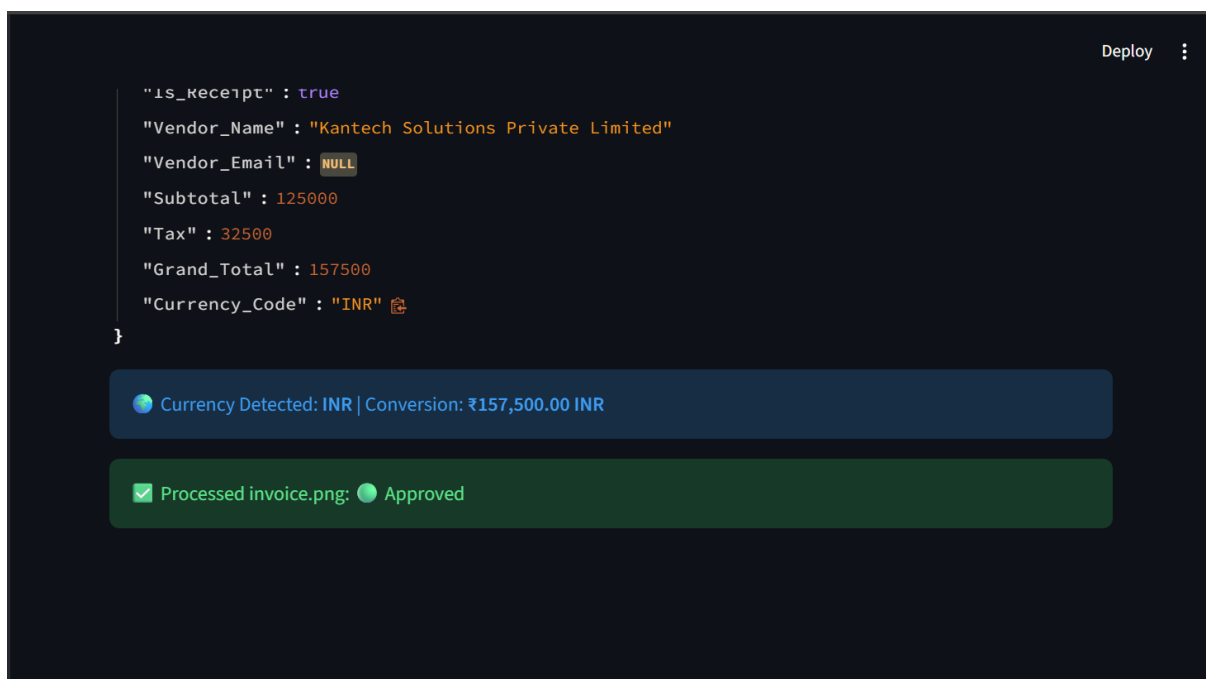


The image shows the 'Multi-Invoice Processing' operator screen. It has a dark background. At the top right, there is a 'Deploy' button with a three-dot menu icon. On the left side, there is a sidebar with a user profile icon and the text 'user'. Below this, there is a 'Navigate' section with two options: 'Upload Invoice' (selected with a red dot) and 'Communication Hub'. At the bottom of the sidebar, there is a 'Logout' button. The main area of the screen has the heading 'Multi-Invoice Processing' with a document icon. Below the heading, there is a section titled 'Upload Invoices'. It contains a large box with the text 'Drag and drop files here' and 'Limit 200MB per file • PNG, JPG, PDF, JPEG'. To the right of this box is a 'Browse files' button.

3.8.2 -INPUT SCREEN



OUTPUT SCREEN-



3.8.3 OPERATORS COMMUNICATION SECTION

The screenshot shows the 'Communication Hub' interface for a user. The left sidebar contains a user profile 'user', a 'Navigate' section with 'Upload Invoice' and 'Communication Hub' (selected), and a 'Logout' button. The main content area has a 'Deploy' button in the top right. The title 'Communication Hub' is followed by a status 'Automated Email Sent' with a message icon and the text 'No vendors require email follow-up.' Below this is a section 'Manual Phone Call Needed' with a list of contacts, showing 'Contact: testing1'.

3.8.4 ADMIN SCREEN :

The screenshot shows the 'Master Invoice Ledger' interface for an admin. The left sidebar contains an 'admin' profile, a 'Navigate' section with 'Manage Invoices' (selected), 'Security Audit', and 'Analytics Dashboard', and a 'Logout' button. The main content area has a 'Deploy' button in the top right. The title 'Master Invoice Ledger' is followed by a table with 8 columns: filename, vendor_name, Original Amount, currency, Amount pay(INR), status, and proces. The table contains 4 rows of data. Below the table are two buttons: 'Download Formatted CSV' and 'Download Formatted Excel'.

	filename	vendor_name	Original Amount	currency	Amount pay(INR)	status	proces
1	Screenshot 2026-04-16 003339.png	testing1	230	INR	230	Approved	user
2	Screenshot 2026-04-16 002552.png	testing1	220	INR	220	Approved	user
3	Screenshot 2026-04-15 235037.png	testing1	220	CNY	2975.85	Approved	user
4	Screenshot 2026-04-15 233708.png	testing1	200	JPY	None	Approved	user

3.8.5 ADMIN SECURITY LOG

admin

Manage Invoices

Security Audit

Analytics Dashboard

Logout

Security Audit Log

	username	action	timestamp
0	user	Logged In	2026-03-27 23:51:08
1	admin	Logged In	2026-03-27 23:49:08
2	admin	Logged In	2026-03-27 23:45:33
3	user	Logged In	2026-03-27 23:44:29
4	admin	Logged In	2026-03-25 23:40:05
5	admin	Logged In	2026-03-25 23:36:44
6	user	Logged In	2026-03-25 23:34:54
7	admin	Logged In	2026-03-25 23:21:15
8	user	Logged In	2026-03-25 23:20:06
9	user	Logged In	2026-03-25 23:19:12

3.8.6 ANALYTICS DASHBOARD

admin

Manage Invoices

Security Audit

Analytics Dashboard

Logout

Financial Analytics Dashboard

Filter by Month

All Time

Spending (All Time)

₹3,665.85

Invoices Processed

5

Approval Rate

80.0%

Vendor Distribution

5

4

Chapter 4: Coding

The implementation phase involves converting the system design into a working application using programming languages and tools. The FINOPS system is developed using Python as the core language, with Streamlit for the user interface and MySQL for database management. The implementation focuses on automating invoice processing, validation, and data storage while ensuring security and efficiency.

The system is divided into multiple modules, each responsible for a specific functionality. These modules are described below along with their implementation details.

4.1 Algorithm

In this section, we define the step-by-step logical sequence that our system follows to process an invoice.

Algorithm 1: The Intelligent Invoice Processing Pipeline

This algorithm acts as the system's primary entry point, functioning as the gatekeeper, reader, and auditor for every financial document.

- Step 1 [Initialization]: The system listens for a file upload (PNG, JPG, or PDF) through the `st.file_uploader` interface.
- Step 2 [Cryptographic Hashing]: * The application reads the raw file bytes.
 - It generates a unique 32-character MD5 hash using `hashlib.md5` to create a digital fingerprint, ensuring data integrity.
- Step 3 [Duplicate Verification]: * The system executes a query against the MySQL `invoices` table to check if the current `file_hash` already exists.
 - If match found: The process is aborted with a "Duplicate Document" warning to prevent double-billing.
 - If no match: The pipeline proceeds to extraction.

- Step 4 [AI Data Extraction]: * The file is transmitted to the Google Gemini Vision API.
 - The AI parses the document and returns a structured JSON object containing: Vendor_Name, Vendor_Email, Subtotal, Tax, Grand_Total, and Currency_Code.
- Step 5 [Mathematical & Policy Audit]: * The system performs a server-side calculation: $\text{Audit_Sum} = \text{Subtotal} + \text{Tax}$.
 - Logic Gate: If $\text{abs}(\text{Audit_Sum} - \text{Grand_Total}) < 0.05$, the invoice status is set to "🟢 Approved"; otherwise, it is set to "🔴 Flagged" due to a math discrepancy.
- Step 6 [Database Persistence]: * The system invokes Algorithm 3 to determine the correct INR value.
 - All extracted fields, the original currency, the unique hash, and the converted amount_inr are saved into the invoices table.
- Step 7 [Termination]: The operator's dashboard renders the extracted JSON and the final status indicator.

Algorithm 2: Automated Dispute Resolution (Email Agent)

This algorithm handles communication workflows when discrepancies are detected during the audit phase.

- Step 1 [Fetch Context]: The system retrieves the flagged invoice metadata, specifically the vendor_name, vendor_email, and the calculated discrepancy amount.
- Step 2 [Validation]: * If Vendor_Email is present: The system proceeds to automated drafting.
 - If Vendor_Email is null/empty: The vendor is routed to the "Manual Phone Call Needed" section of the Communication Hub for operator intervention.
- Step 3 [Drafting]: A professional dispute email is dynamically generated, including details of the mathematical error and a request for a corrected invoice.
- Step 4 [Transmission]: The email_service module establishes a secure connection to the SMTP server (Gmail) using TLS encryption.

- Step 5 [Execution & Logging]: * The email is dispatched to the vendor.
 - A record is inserted into the `audit_logs` table, noting that a dispute email was sent for the specific invoice.

Algorithm 3: Multi-Currency Financial Synchronizer

This logic ensures that transactions in global currencies (JPY, CNY, USD, GBP) are accurately converted to a base currency (INR) and stored as fixed values to prevent future recalculation errors.

- Step 1 [Currency Detection]: The `Currency_Code` (or symbol like ₹, \$, £) is received from the AI extraction output.
- Step 2 [Standardization]: Common markers (e.g., "rs.", "rs", "₹", "inr") are converted to a standardized lowercase format for logic branching.
- Step 3 [Domestic 1:1 Branching]: If the detected code is an Indian marker, the multiplier is set to 1.0 to ensure no conversion is applied to local receipts.
- Step 4 [Dynamic Conversion Branching]: * The system attempts to fetch a real-time exchange rate via the `currency_converter` library.
 - Fallback Logic: If the API is unreachable, the system uses pre-defined 2026 rates: USD: 83.0, GBP: 125.0, CNY: 11.5, JPY: 0.58.
- Step 5 [Calculation]: The final value is computed: $\text{amount_inr} = \text{float}(\text{Grand_Total}) * \text{multiplier}$.
- Step 6 [Persistence]: The system saves this calculated `amount_inr` into its own dedicated database column, ensuring the Admin Panel always displays the exact value the operator saw.

Algorithm 4: Administrative Governance & Audit Reporting

This algorithm provides the administrative layer with oversight tools and formatted data exports for external accounting.

- Step 1 [Role Authentication]: The system verifies `st.session_state['role'] == 'Admin'` before granting access to ledger and audit views.
- Step 2 [Data Aggregation]: * A SQL `JOIN` query is executed to combine invoice data with the `username` of the operator who processed it.
 - Audit logs are fetched and sorted by `timestamp DESC` to show the most recent actions first.
- Step 3 [UI Rendering]: Data is displayed using `st.dataframe` with `use_container_width=True`, which prevents UI overflow and ensures readability for long vendor names or timestamps.
- Step 4 [Report Formatting]: * CSV Path: Data is converted to a `StringIO` buffer to maintain proper column alignment when opened in Excel.
 - Excel Path: The `openpyxl` engine calculates the maximum length of data in each column and automatically adjusts cell widths (`Length + 2`) for a professional look.
- Step 5 [Analytics Summation]: The "Total Spending" metric is calculated by performing a `SUM` directly on the `amount_inr` database column for the selected month, ensuring that multi-currency totals are accurately aggregated in Rupees

4.2 CODE SNIPPETS :

4.2.1 DATABASE CONNECTION :

```
import mysql.connector
import os
from dotenv import load_dotenv
# Load secrets from .env file
load_dotenv()
def get_db_connection():
    """Establishes and returns a connection to the MySQL database."""
    try:
        conn = mysql.connector.connect(
            host=os.getenv("DB_HOST"),
            user=os.getenv("DB_USER"),
            password=os.getenv("DB_PASSWORD"),
            database=os.getenv("DB_NAME")
        )
        return conn
    except mysql.connector.Error as err:
        print(f'Error connecting to database: {err}')
        return None

def initialize_database():
    """Creates the 3 core tables if they do not exist."""

    # First, connect without a database to create the DB itself if needed
    temp_conn = mysql.connector.connect(
        host=os.getenv("DB_HOST"),
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD")
    )
```

```

temp_cursor = temp_conn.cursor()
temp_cursor.execute(
    f'CREATE DATABASE IF NOT EXISTS {os.getenv('DB_NAME')}}')
temp_conn.close()

# Now connect to the specific database
conn = get_db_connection()
if conn is None:
    return

cursor = conn.cursor()

if __name__ == "__main__":
    initialize_database()

```

4.2.2 : DISPUTE EMAIL PROGRAM:

```

import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
import os
from dotenv import load_dotenv

# Load your email credentials from .env
load_dotenv()

def send_dispute_email(vendor_email, vendor_name, subtotal, tax, grand_total):
    """Sends an automated email to the vendor if calculations are wrong."""

    # If the AI couldn't find an email on the invoice, we can't send a dispute
    if not vendor_email or vendor_email == "":
        return False, "No vendor email found on the invoice."

```

```
sender_email = os.getenv("EMAIL_SENDER")
sender_password = os.getenv("EMAIL_PASSWORD")
```

```
# Calculate what the total SHOULD be
expected_total = float(subtotal) + float(tax)
```

```
# Create the email message
msg = MIMEMultipart()
msg['From'] = sender_email
msg['To'] = vendor_email
msg['Subject'] = "Invoice Discrepancy Notice - Action Required"
```

```
# The body of the email
```

```
body = f"""
```

```
Dear {vendor_name},
```

We are processing your recent invoice, but our automated system flagged a calculation error.

Details extracted from your invoice:

- Subtotal: \${subtotal}
- Tax: \${tax}
- Stated Grand Total: \${grand_total}

However, our system calculated that Subtotal + Tax equals \${expected_total:.2f}.

Please review your billing system and send a corrected invoice so we can process your payment promptly.

Regards,

FinOps Automation System

```
"""
```



```

msg.attach(MIMEText(body, 'plain'))

try:
    # Connect to Google's SMTP Server
    server = smtplib.SMTP('smtp.gmail.com', 587)
    server.starttls() # Secure the connection
    # Login using your App Password
    server.login(sender_email, sender_password)

    # Send the email!
    server.send_message(msg)
    server.quit()
    return True, f'Dispute email sent to {vendor_email}.'
except Exception as e:
    return False, f'Failed to send email: {e}'

```

4.3.3 : APP PROGRAM

```

import streamlit as st
import pandas as pd
import hashlib
import io
from currency_converter import CurrencyConverter
from database import get_db_connection
from auth import authenticate_user
from ai_engine import extract_invoice_data
from email_service import send_dispute_email

# --- PAGE CONFIG ---

```

```
st.set_page_config(page_title="FinOps AI", page_icon="🧠", layout="wide")
```

```
# --- EXCEL FORMATTER ---
```

```
def to_excel(df):
    output = io.BytesIO()
    with pd.ExcelWriter(output, engine='openpyxl') as writer:
        df.to_excel(writer, index=False, sheet_name='Monthly_Report')
        worksheet = writer.sheets['Monthly_Report']
        for col in worksheet.columns:
            max_length = max([len(str(cell.value)) for cell in col])
            worksheet.column_dimensions[col[0]
                                     .column_letter].width = max_length + 2
    return output.getvalue()
```

```
# --- 1. LOGIN SCREEN ---
```

```
def login_screen():
    st.title("🔒 FinOps Login")
    with st.form("login"):
        u = st.text_input("Username")
        p = st.text_input("Password", type="password")
        if st.form_submit_button("Login"):
            user, msg = authenticate_user(u, p)
            if user:
                st.session_state.update(
                    {"logged_in": True, "user_id": user['id'], "user": user['username'], "role":
user['role']})
                st.query_params.update(
                    {"authenticated": "true", "uid": user['id'], "uname": user['username'], "urole":
user['role']})
                st.rerun()
```

```

else:
    st.error(msg)

# --- 2. ADMIN PAGES ---
def admin_manage_invoices():
    st.header("📄 Master Invoice Ledger")
    conn = get_db_connection()
    df = pd.read_sql("""
        SELECT i.filename, i.vendor_name, i.grand_total as 'Original Amount',
        i.currency, i.amount_inr as 'Amount pay(INR)', i.status,
        u.username as processed_by, i.upload_time
        FROM invoices i
        JOIN users u ON i.uploaded_by = u.id
        ORDER BY i.upload_time DESC""", conn)
    conn.close()
    if not df.empty:
        approved_only_df = df[df['status'] == '🟢 Approved'].copy()
        if not approved_only_df.empty:
            st.dataframe(approved_only_df, use_container_width=True)

        st.divider()
        col1, col2 = st.columns(2)
        with col1:
            csv_buffer = io.StringIO()
            approved_only_df.to_csv(csv_buffer, index=False)
            st.download_button("Download Formatted CSV", csv_buffer.getvalue(
            ), "formatted_payments.csv", "text/csv")
        with col2:
            st.download_button("Download Formatted Excel", to_excel(
                approved_only_df, "Monthly_Report.xlsx", "application/vnd.openxmlformats-
                officedocument.spreadsheetml.sheet")

```

```

else:
    st.info("No invoices processed yet.")

```

```

def admin_audit():
    st.header("🕵️ Security Audit Log")
    conn = get_db_connection()
    df = pd.read_sql("""
        SELECT u.username, a.action, a.timestamp
        FROM audit_logs a
        JOIN users u ON a.user_id = u.id
        ORDER BY a.timestamp DESC""", conn)
    conn.close()
    st.dataframe(df, use_container_width=True)

```

```

def admin_analytics():
    st.header("📊 Financial Analytics Dashboard")
    conn = get_db_connection()
    df = pd.read_sql(
        "SELECT amount_inr, status, vendor_name, upload_time FROM invoices", conn)
    conn.close()

    if not df.empty:
        df['upload_time'] = pd.to_datetime(df['upload_time'])
        df['month_year'] = df['upload_time'].dt.to_period('M').astype(str)
        available_months = sorted(df['month_year'].unique(), reverse=True)
        selected_month = st.selectbox(
            "📅 Filter by Month", ["All Time"] + available_months)

        filtered_df = df.copy()
        if selected_month != "All Time":

```

```

        filtered_df = df[df['month_year'] == selected_month]

        c1, c2, c3 = st.columns(3)
        total_spent_inr = filtered_df['amount_inr'].sum()
        c1.metric(f'Spending ({selected_month})', f'₹{total_spent_inr:,.2f}')
        c2.metric("Invoices Processed", len(filtered_df))
        approved_count = len(
            filtered_df[filtered_df['status'] == '🟢 Approved'])
        acc = (approved_count / len(filtered_df)) * \
            100 if len(filtered_df) > 0 else 0
        c3.metric("Approval Rate", f'{acc:.1f}%')

        st.divider()
        st.subheader("Vendor Distribution")
        st.bar_chart(filtered_df['vendor_name'].value_counts())
    else:
        st.info("No data available.")

# --- 3. OPERATOR PAGES ---
def operator_upload():
    st.header("📁 Multi-Invoice Processing")
    uploaded_files = st.file_uploader(
        "Upload Invoices", type=['png', 'jpg', 'pdf'], accept_multiple_files=True)

    if uploaded_files:
        for f in uploaded_files:
            file_bytes = f.getvalue()
            file_hash = hashlib.md5(file_bytes).hexdigest()

            with st.spinner(f"Processing {f.name}..."):
                data = extract_invoice_data(file_bytes, f.type)

```

```

if data:
    st.write(f'### {f.name}')
    st.json(data)

if not data or data.get("Is_Receipt") is False:
    st.error(
        f"❌ Rejected: '{f.name}' is not a valid receipt/invoice.")
    continue

curr = data.get("Currency_Code") or data.get(
    "Currency") or "USD"
total = float(data.get("Grand_Total", 0))

# Dynamic check for Indian Rupee
total_inr = get_dynamic_inr(total, curr)

sub, tax = float(data.get('Subtotal', 0)
    ), float(data.get('Tax', 0))
status = "🟢 Approved" if abs(
    (sub + tax) - total) < 0.05 else "🔴 Flagged"

try:
    conn = get_db_connection()
    cur = conn.cursor()
    cur.execute("""
        INSERT INTO invoices (filename, file_hash, vendor_name, vendor_email,
grand_total, status, uploaded_by, currency, amount_inr)
        VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)""",
        (f.name, file_hash, data['Vendor_Name'], data['Vendor_Email'], total,
status, st.session_state['user_id'], curr, total_inr))
    conn.commit()

```

```

cur.close()
conn.close()

if "Flagged" in status and data['Vendor_Email']:
    send_dispute_email(
        data['Vendor_Email'], data['Vendor_Name'], sub, tax, total)

st.info(
    f"🌐 Currency Detected: **{curr}** | Conversion: **₹{total_inr:,.2f} INR**")
st.success(f"✅ Processed {f.name}: {status}")

except Exception as e:
    if "1062" in str(e):
        st.warning(
            f"⚠️ Skipping '{f.name}': Already processed.")
    else:
        st.error(f"Error saving {f.name}: {e}")

def operator_comms():
    st.header("📞 Communication Hub")
    conn = get_db_connection()
    df = pd.read_sql(
        "SELECT vendor_name, vendor_email FROM invoices WHERE status LIKE '%Flagged%'", conn)
    conn.close()

    if df.empty:
        st.info("No follow-ups needed.")
    else:
        st.subheader("📧 Automated Email Sent")
        email_df = df[df['vendor_email'].str.strip() != ""]

```

```

if email_df.empty:
    st.write("No vendors require email follow-up.")
else:
    for i, row in email_df.iterrows():
        with st.expander(f'Contact: {row["vendor_name"]}'):
            st.success(f'***Email Sent To:** {row["vendor_email"]}')
            st.write(
                "Dispute details have been automatically mailed to the vendor.")

st.divider()

st.subheader("📞 Manual Phone Call Needed")
phone_df = df[df['vendor_email'].str.strip() == ""]
if phone_df.empty:
    st.write("No manual calls required.")
else:
    for i, row in phone_df.iterrows():
        with st.expander(f'Contact: {row["vendor_name"]}'):
            st.error("❌ No email found on receipt.")
            st.write(
                "***Action:** Please contact this vendor manually to clarify the flagged
discrepancy.")

# --- MAIN NAVIGATION ---
params = st.query_params
if 'logged_in' not in st.session_state:
    if params.get("authenticated") == "true":
        st.session_state.update({"logged_in": True, "user_id": int(params.get(
            "uid")), "user": params.get("uname"), "role": params.get("urole")})
    else:
        st.session_state['logged_in'] = False

```



```
if not st.session_state['logged_in']:
    login_screen()
else:
    with st.sidebar:
        st.title(f"👤 {st.session_state['user']}")
```

```
if page == "Manage Invoices":
    admin_manage_invoices()
elif page == "Security Audit":
    admin_audit()
elif page == "Analytics Dashboard":
    admin_analytics()
elif page == "Upload Invoice":
    operator_upload()
elif page == "Communication Hub":
    operator_comms()
```

Chapter 5: Testing

5.1 Test Strategy

Testing a system that involves both Artificial Intelligence and strict financial rules requires a hybrid approach. Our testing strategy was divided into two main phases to ensure both the logic and the user experience were flawless:

Testing a system that integrates Generative AI (Gemini) with strict financial conversion rules requires a hybrid approach. Our strategy ensured that the non-deterministic nature of AI and the fixed requirements of accounting were both validated. The testing was divided into three core phases:

- Logical Extraction Validation: Ensuring the AI correctly identifies metadata (Vendor, Total, Currency) from diverse document layouts.
- Financial Integrity Testing: Verifying that currency conversion (especially the 1:1 INR rule) remains consistent between the Operator's view and the Admin's database ledger.
- Workflow Acceptance: Validating the end-to-end process from file upload to automated or manual vendor communication.

5.2 Unit Test Plan

Unit testing verified the smallest independent parts of the application to ensure they worked perfectly before integration into the dashboard.

Multi-Currency Conversion Module (`get_dynamic_inr`): We passed simulated data to test the conversion logic:

Foreign Currency Case: Inputting "210" with "JPY" verified the system used the live or fallback rate to return approximately 123.47.

Duplicate Detection Module: We passed the same receipt image through the `hashlib.md5` function 10 times to ensure it generated the exact same unique 32-character fingerprint.

Data Persistence Module: We verified that the new amount_inr column successfully captured the final converted value at the moment of upload, ensuring the Admin Panel fetches a fixed value rather than recalculating it.

5.3 Acceptance Test Plan (UAT)

User Acceptance Testing ensured the system solved real business problems for the two target roles:

Operator Acceptance: Verified that an operator can upload an invoice, view the AI-extracted JSON, and see a clear conversion summary in INR regardless of the original currency.

Admin Acceptance: Verified that the Finance Manager can view the Master Invoice Ledger where the amount_inr column matches the operator's display exactly, and successfully download formatted CSV/Excel files for accounting.

Workflow Acceptance: Verified that if a receipt lacks an email address, the vendor is correctly diverted to the "Manual Phone Call Needed" section in the Communication Hub.

5.4 Test Case / Test Script

Here is a snapshot of our formal test scripts. We created structured test cases to systematically check the core functional requirements.

Test Case ID	Feature Tested	Action Performed (Input)	Expected Result	Actual Result	Status
TC-01	Duplicate Prevention	Upload a receipt image, then upload the same image again	System should block the second upload and display "Skipping: Already processed."	System blocked the duplicate based on file hash	PASS
TC-02	AI Extraction Accuracy	Upload a clear image with vendor "testing1" and total 180	System should correctly extract vendor name and total amount	Vendor name and total extracted	PASS

Test Case ID	Feature Tested	Action Performed (Input)	Expected Result	Actual Result	Status
				correctly using AI	
TC-03	Math Validation	Upload invoice where Subtotal = 180, Tax = 30, Total = 150	System should flag the invoice due to incorrect calculation	Invoice flagged and dispute process triggered	PASS
TC-04	Role-Based Access	Log in as Operator and try accessing Admin features	System should restrict access to Admin modules	Admin modules were hidden for Operator	PASS
TC-05	INR 1:1 Sync	Upload receipt with “Rs.180.00”	System should store and display ₹180.00 correctly	₹180.00 stored and displayed accurately	PASS
TC-06	Multi-Currency Sync	Upload receipt with “210 JPY”	System should convert and store correct INR value	Converted value stored and displayed correctly	PASS
TC-07	Communication Handling	Upload receipt without vendor email	System should mark for manual follow-up	Invoice categorized correctly for manual action	PASS
TC-08	Analytics Dashboard	View reports after multiple uploads	System should correctly calculate total spending	Dashboard displayed accurate total values	PASS
TC-09	Data Export	Click “Download CSV” option	Exported file should have proper formatting	CSV file generated with correct structure	PASS
TC-10	UI Handling	Open audit logs with long usernames	Interface should handle display without overflow	UI remained responsive and properly formatted	PASS

Chapter 6: Limitations of the Proposed System

Every software system has its boundaries, and acknowledging them is a crucial part of professional software engineering. Although the FINOPS system provides automation, accuracy, and efficiency in invoice processing, it still has certain limitations. These limitations arise due to dependency on external services, system design constraints, and the current scope of implementation. The key limitations of the proposed system are described below:

- **Dependency on Internet Connectivity**

The system relies on cloud-based services such as AI data extraction and email communication. Therefore, a stable internet connection is required for proper functioning. In the absence of internet connectivity, critical features like invoice processing and email sending cannot be executed.

- **Dependence on AI Accuracy**

The system uses AI (Gemini API) for extracting data from invoices. While the accuracy is generally high, it may produce incorrect or incomplete results when the invoice format is unclear, handwritten, or poorly scanned. This may require manual verification by the Operator.

- **Limited Handling of Complex Invoice Formats**

The system is designed to handle standard invoice structures. If invoices contain complex layouts, multiple currencies in one document, or unconventional formats, the extraction and validation process may not work as expected.

- **Basic Validation Mechanism**

The system currently performs only mathematical validation and basic policy checks. It does not support advanced financial validations such as tax rules, vendor verification, or cross-referencing with external systems.

- **No Direct Integration with Banking Systems**

The system does not perform actual financial transactions such as payments through banking systems (NEFT/RTGS). It is limited to invoice processing and validation only.

- **Limited Role Management**

The system supports only two roles: Operator and Administrator. It does not include additional roles such as auditors or finance analysts, which may be required in larger organizations.

- **Email Dependency for Dispute Resolution**

The dispute resolution process depends entirely on email communication. If the vendor email is missing or incorrect, the system cannot automatically resolve the issue and requires manual intervention.

- **Scalability Constraints for Large Enterprises**

Although the system can handle moderate workloads, it may require optimization and infrastructure upgrades to handle very large-scale enterprise-level data efficiently.

- **No Offline Functionality**

The system is entirely web-based and cannot function in offline mode. All operations require active connectivity to the database and external services.

The above limitations highlight areas where the FINOPS system can be improved in future versions. Addressing these limitations will enhance the system's performance, scalability, and reliability, making it more suitable for large-scale and complex financial environments.

Chapter 7: Proposed Enhancements

The FINOPS system provides an efficient and automated solution for invoice processing; however, there are several opportunities for enhancement and expansion in future versions. With advancements in technology and increasing business requirements, the system can be further improved to provide more intelligent, scalable, and secure financial operations. The potential future enhancements are described below:

- **Enhanced AI Accuracy and Learning**

The system can be improved by integrating more advanced AI models capable of handling complex and unstructured invoice formats. Future versions can include self-learning capabilities, where the system improves its accuracy over time by learning from previously processed invoices.

- **Support for Multiple Invoice Formats**

Currently, the system performs best with standard invoice formats. In the future, it can be enhanced to handle diverse invoice structures, including handwritten invoices, multi-page documents, and region-specific formats.

- **Advanced Validation Mechanisms**

The validation process can be extended to include advanced financial checks such as tax compliance, vendor verification, and cross-validation with historical data. This will further improve accuracy and reduce risks.

- **Integration with ERP Systems**

The system can be integrated with existing enterprise systems such as accounting or ERP platforms. This will enable seamless data transfer between systems and reduce the need for manual intervention.

- **Improved Security Features**

Future enhancements may include advanced security mechanisms such as multi-factor authentication, data encryption, and secure API communication. These features will strengthen system security and protect sensitive financial data.

- **Mobile Application Support**

The system can be extended to a mobile platform, allowing users to upload invoices and monitor processing through mobile devices. This will improve accessibility and convenience.

- **Real-Time Notifications and Alerts**

The system can be enhanced to provide real-time notifications for important events such as invoice approval, rejection, or email delivery status. This will improve user awareness and responsiveness.

- **Scalability for Large Enterprises**

Future versions can be optimized to handle large-scale data processing with improved performance. This includes better database optimization and support for high-volume transactions.

- **Multi-Language Support**

The system can be extended to support multiple languages for invoice processing and communication. This will make it suitable for global organizations.

- **Advanced Reporting and Analytics**

The analytics module can be enhanced to include detailed financial insights, predictive analysis, and customizable reports. This will help management in better decision-making.

The future scope of the FINOPS system focuses on enhancing intelligence, scalability, and usability while maintaining accuracy and efficiency. These improvements will transform the system into a more powerful and adaptable solution for modern financial operations.

Chapter 8: Conclusion

The FINOPS – Automated Intelligent Document Processing System for Financial Operations successfully addresses the challenges associated with traditional Accounts Payable processes. In existing systems, invoice handling is largely manual, time-consuming, and prone to human errors, which often leads to inefficiencies, delays, and financial risks such as duplicate payments and incorrect data entry.

The proposed system introduces automation and intelligence into the invoice processing workflow by integrating modern technologies such as AI-based data extraction, cryptographic hashing, and structured database management. By eliminating manual data entry and automating validation processes, the system significantly improves accuracy and reduces processing time. The implementation of MD5 hashing ensures that duplicate invoices are detected and prevented, thereby minimizing financial losses.

The system also enhances reliability by performing automatic validation of invoice data, including mathematical checks and policy compliance. In case of discrepancies, the automated email mechanism streamlines communication with vendors, reducing manual effort and improving response time. This makes the overall workflow faster and more efficient.

Furthermore, the use of Role-Based Access Control ensures secure and controlled access to system functionalities. Operators are responsible for invoice processing, while administrators monitor system activities and analyze financial data through dashboards and reports. The inclusion of audit logs provides transparency and accountability by maintaining a record of all user actions.

The FINOPS system is designed to be scalable and adaptable, allowing it to handle increasing volumes of invoices while maintaining performance. Its modular architecture and use of widely available technologies make it easy to maintain and extend in the future.

However, certain limitations such as dependency on internet connectivity and AI accuracy have been identified, which provide opportunities for further improvement. The future scope of the system includes enhancements such as advanced validation mechanisms, improved AI capabilities, and integration with enterprise systems.

In conclusion, the FINOPS system provides an effective and reliable solution for automating financial operations in the Accounts Payable domain. It reduces manual workload, improves data accuracy, enhances security, and supports better decision-making. The successful implementation of this system demonstrates the potential of combining automation and artificial intelligence to transform traditional business processes into efficient and intelligent systems.

Chapter 9: Bibliography

References:

9.1 Core Project Frameworks and Languages

- Python Software Foundation. (2026). *Python 3.9 Language Reference*. Retrieved from <https://docs.python.org/3.9/> (Standard for core logic, smtplib for disputes, and hashlib for duplicate detection).
- Streamlit Inc. (2026). *Streamlit Documentation: The Fastest Way to Build Data Apps*. Retrieved from <https://docs.streamlit.io/> (Reference for the multi-page dashboard, sidebar navigation, and session state management).
- Pandas Development Team. (2026). *Pandas: High-performance data structures and data analysis tools*. Retrieved from <https://pandas.pydata.org/> (Reference for the Master Ledger and CSV/Excel data processing).

9.2 Artificial Intelligence and Conversion Engines

- Google DeepMind. (2026). *Gemini 1.5 Flash Vision API Documentation*. Retrieved from <https://ai.google.dev/docs> (Primary reference for the AI multimodal engine used for receipt metadata extraction).
- CurrencyConverter Team. (2026). *CurrencyConverter 0.17.x Documentation*. Retrieved from <https://pypi.org/project/CurrencyConverter/> (Reference for the dynamic JPY, CNY, and GBP to INR conversion logic).
- ISO 4217 Standard. (2026). *Codes for the representation of currencies*. International Organization for Standardization (Standard used to implement standardized currency detection for Rupees and Dollars).

9.3 Data Management :

- Oracle Corporation. (2026). *MySQL 8.0 Reference Manual*. Retrieved from <https://dev.mysql.com/doc/> (Reference for financial ledger schema, 3NF normalization, and column management for amount_inr and currency).
- OpenPyXL Development Team. (2026). *OpenPyXL - A Python library to read/write Excel 2010 xlsx/xlsm files*. (Reference used for the formatted Excel download and automatic column width adjustment).
- NumPy Developers. (2026). *NumPy Reference Guide*. (Consulted for high-precision floating-point arithmetic in the subtotal and tax audit module).

9.4 Security and Communication Standards

- IETF (Internet Engineering Task Force). (2011). *The MD5 Message-Digest Algorithm - RFC 1321*. (Standard consulted for creating unique digital fingerprints of uploaded files for duplicate prevention).
- TLS Protocol. (2026). *Transport Layer Security (TLS) for Secure Email Transmission*. (Standard used for the dispute agent connecting to Gmail's SMTP servers).
- OWASP Foundation. (2026). *OWASP Top 10 Web Application Security Risks*. (Referenced for implementing role-based access control (RBAC) and data sanitization in the admin panel).

9.5 Software Engineering Methodology

- Pressman, R. S., & Maxim, B. R. (2014). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education. (Standard reference for documenting the system requirements, ERD diagrams, and the SDLC methodology of this project).
- Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson Education. (Referenced for modular software design and the separation of AI extraction from dashboard UI logic).

Annexure: Progress Sheet

FINOPS Application

User Manual

1. Introduction

The FINOPS – Automated Intelligent Document Processing System is designed to simplify and automate financial operations, particularly in the Accounts Payable domain. This User Manual provides a comprehensive and structured guide for understanding and operating the system effectively. It explains the features, functionalities, and step-by-step procedures required to use the application.

The primary objective of this system is to eliminate manual invoice processing by introducing automation through Artificial Intelligence. The system is capable of extracting invoice data, validating financial details, detecting duplicate entries, and maintaining structured records in a centralized database. By automating these processes, the system reduces human effort, minimizes errors, and improves overall efficiency.

This manual is intended for two main types of users:

Operators, who are responsible for uploading invoices, reviewing extracted data, and initiating communication in case of discrepancies.

Administrators, who oversee the system, monitor financial activities, analyze reports, and ensure that operations are functioning correctly.

The document provides detailed instructions on how to use each module of the system, including login procedures, invoice processing, validation checks, and reporting features. It serves as a complete reference for users to operate the FINOPS system in a structured and efficient manner.

2. System Overview

The FINOPS system is an intelligent and automated platform designed to streamline invoice processing by integrating Artificial Intelligence, data validation mechanisms, and database management. It replaces traditional manual processes with a structured and efficient workflow.

The system performs several key operations, which are described below:

- **AI-Based Data Extraction:**

The system uses advanced AI technology to extract relevant information from invoice documents. This includes details such as vendor name, vendor email, invoice date, subtotal, tax amount, and total amount. Unlike traditional methods, the system can understand different invoice formats and extract accurate data without manual input.

- **Duplicate Detection:**

To prevent duplicate invoice processing, the system generates a unique MD5 hash for each uploaded file. This hash acts as a digital fingerprint and is compared with existing records. If a duplicate is detected, the system immediately stops further processing and notifies the user.

- **Automated Validation:**

The system performs automatic validation checks on extracted data. It verifies whether the total amount is correctly calculated based on subtotal and tax. Additionally, it checks for any policy violations. If any inconsistency is found, the invoice is flagged for review.

- Email Communication:

In cases where errors or discrepancies are detected, the system automatically generates and sends emails to vendors. This ensures quick communication and reduces the need for manual follow-up.

- Centralized Database Management:

All system data, including user details, invoice records, and activity logs, are stored in a structured database. This ensures data consistency, easy retrieval, and long-term record maintenance.

- Analytics Dashboard:

The system provides an interactive dashboard for administrators. It displays key financial insights such as total invoices processed, approved and flagged invoices, and overall spending trends. This helps in better decision-making and monitoring.

Overall, the FINOPS system enhances efficiency, improves accuracy, and reduces manual workload by automating critical financial operations.

3. Getting Started

To ensure smooth operation of the FINOPS system, certain requirements must be fulfilled before using the application. These requirements ensure that the system functions correctly and provides the expected performance.

- **System Requirements:**

The system is designed to run on modern web browsers such as Google Chrome, Microsoft Edge, or Mozilla Firefox. These browsers support the required web technologies and provide a stable environment for running the application. No additional software installation is required, as the system is web-based.

- **User Credentials:**

Each user must have valid login credentials to access the system. These credentials include a username and password, which are assigned by the system administrator. Based on the credentials, users are granted access according to their roles (Operator or Admin).

- **Internet Connectivity:**

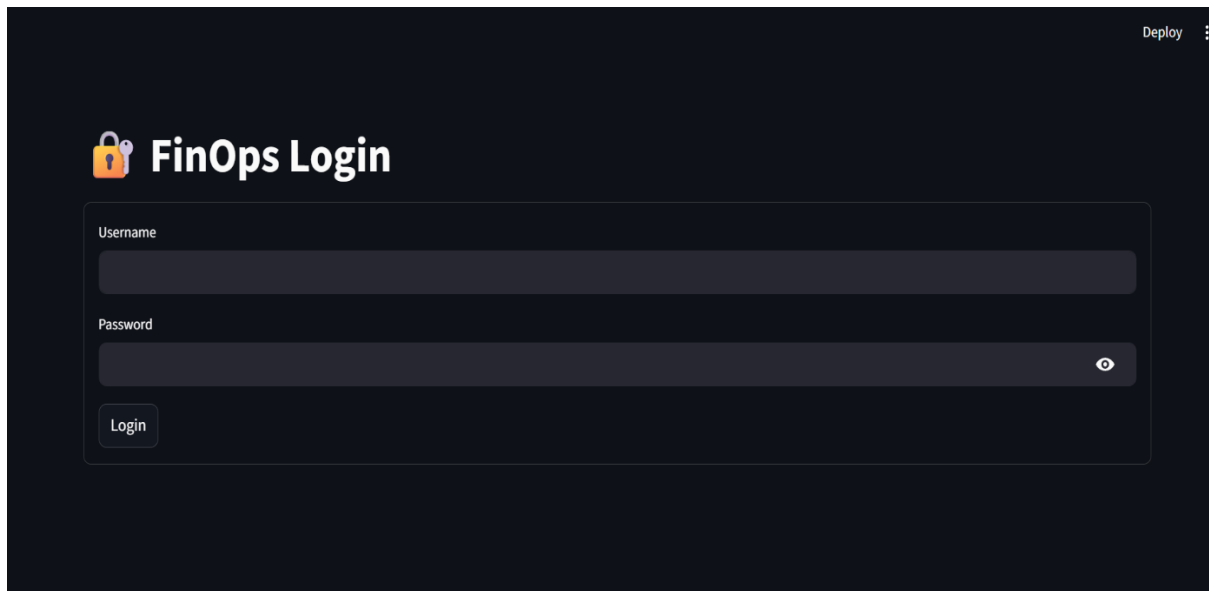
A stable internet connection is essential for the proper functioning of the system. Since the application relies on cloud-based services such as AI data extraction and email communication, uninterrupted connectivity ensures smooth processing and avoids delays.

- **Basic User Knowledge:**

Users should have basic knowledge of using web applications, such as uploading files and navigating dashboards. This helps in understanding the system interface and performing tasks efficiently.

4. Authentication Module

4.1 Login Screen



Purpose:

The login screen acts as the entry point to the system and ensures secure access.

Interface Components:

- Username field
- Password field
- Login button

Working:

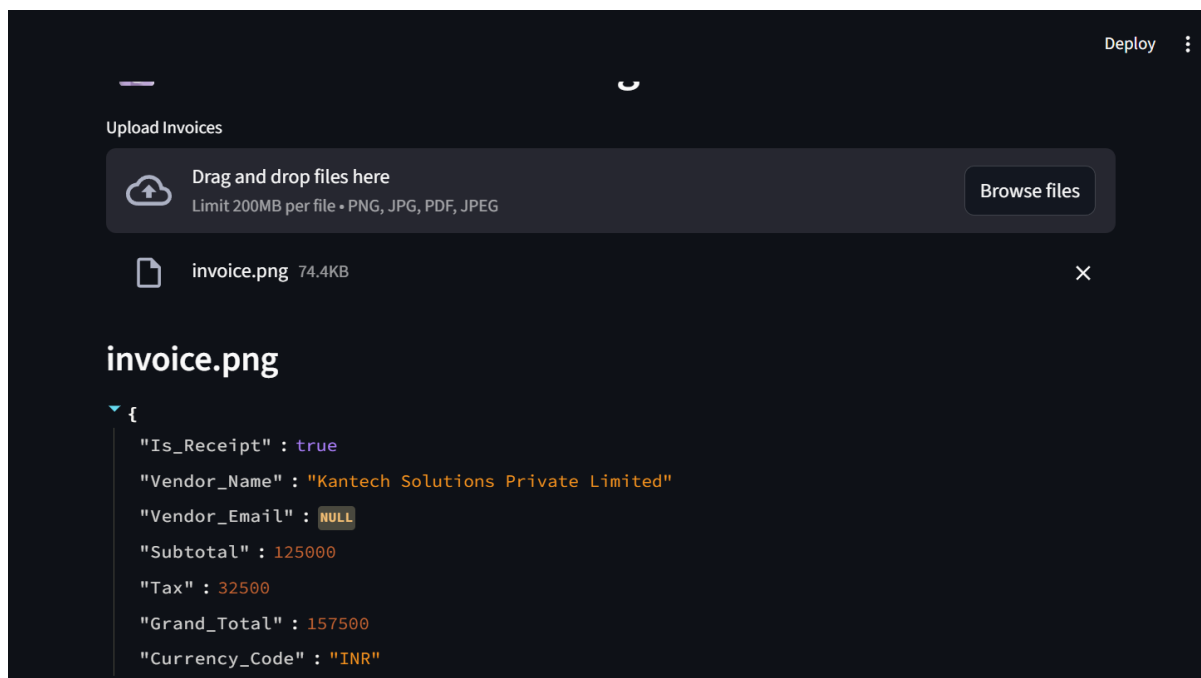
The user enters valid credentials.

The system verifies the information from the database.

Based on the role (Operator/Admin), the user is redirected to the respective dashboard.

5. Operator Module

5.1 Invoice Upload and Processing



Purpose:

To upload and process invoice documents using AI.

Steps:

Select or upload a PDF/image file.

The system generates a unique hash for duplicate detection.

If duplicate → process is stopped.

If valid → data is extracted using AI.

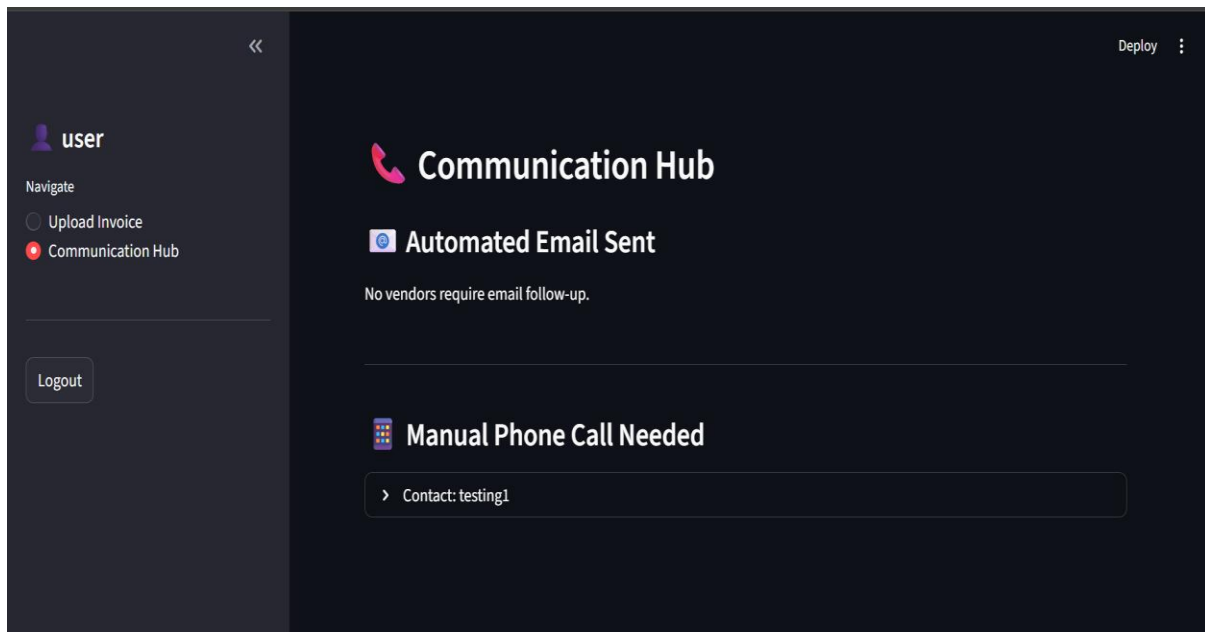
The system performs validation checks.

Final status is displayed (Approved/Flagged)

Output:

Extracted invoice data

5.2 Communication Module



Purpose:

To handle vendor communication for flagged invoices.

Features:

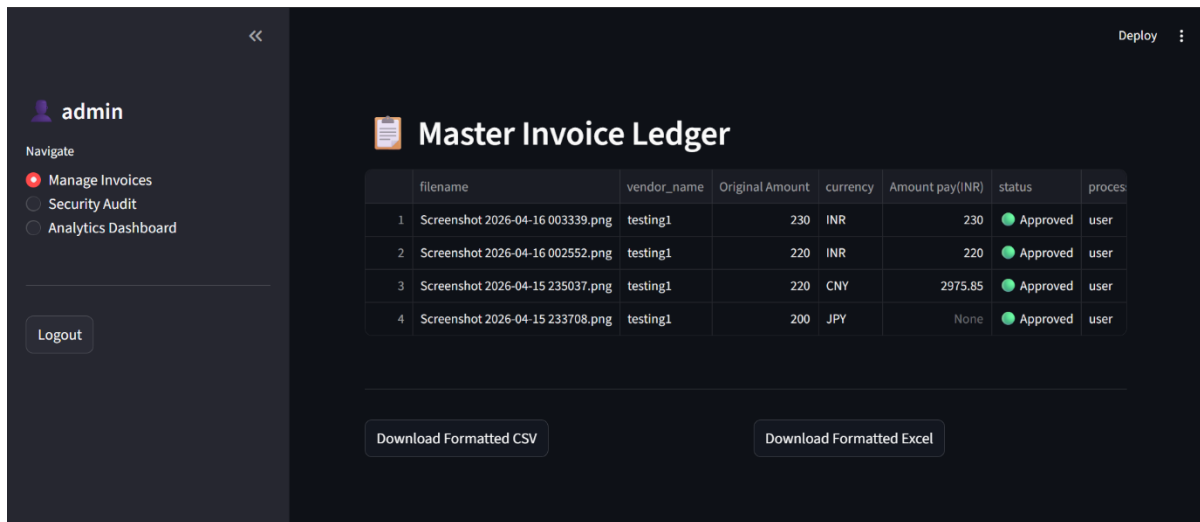
- Automatic email generation
- SMTP-based email sending
- Manual follow-up option if email is not available

Workflow:

- System detects issue
- Email is generated
- Sent to vendor automatically

6. Administrator Module

6.1 Invoice Management Dashboard



	filename	vendor_name	Original Amount	currency	Amount pay(INR)	status	proces
1	Screenshot 2026-04-16 003339.png	testing1	230	INR	230	Approved	user
2	Screenshot 2026-04-16 002552.png	testing1	220	INR	220	Approved	user
3	Screenshot 2026-04-15 235037.png	testing1	220	CNY	2975.85	Approved	user
4	Screenshot 2026-04-15 233708.png	testing1	200	JPY	None	Approved	user

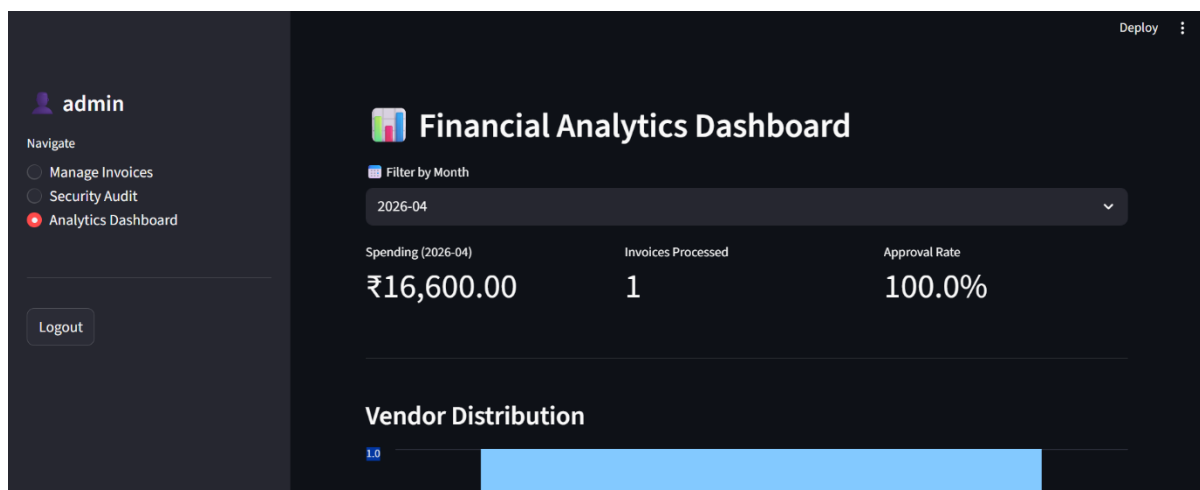
Purpose:

To view and manage all processed invoices.

Features:

- Displays invoice details
- Shows status (Approved/Flagged)
- Tracks uploaded records

6.2 Analytics Dashboard



Purpose:

To provide financial insights.

Features:

- Total invoices processed
- Total spending
- Approved vs flagged ratio
- Visual charts for analysis

7. Glossary

Term	Description
MD5 Hash	Unique identifier generated for files to detect duplicates
AI Extraction	Automated data reading and extraction using Artificial Intelligence
SMTP	Protocol used for sending emails over the internet
Audit Log	Record of system activities performed by users
Invoice Status	Result of validation (Approved or Flagged)
Role-Based Access	Security mechanism that restricts system access based on user roles
Dashboard	Visual interface that displays system data, reports, and analytics